CS 530 Lab Notebook

Lab 10

Name: Jarren Calizo
OdinID: calizo
Date: November 24, 2025

Table of Contents

Lab Overview 4
Lab 10.1g Dataproc 5
Step 1. Dataproc Lab #1 (π) 5
Step 2. Calculating π with random darts 5
Step 3. SparkPi program 5
Step 4. Dataproc setup 6
Step 5. Creating the Dataproc cluster 6
Step 6. Run computation 7
1. How long did the job take to execute? 8
2. Examine output.txt and show the estimate of π calculated. 9
Step 7. Scaling the cluster 9
Step 8. Run computation again 10
1. How long did the job take to execute? How much faster did it take? 11
2. Examine output2.txt and show the estimate of π calculated. 11
Step 9. Cleanup 12
Lab 10.2g Dataflow 13
Step 1. Dataflow Lab #1 - Java package popularity 13
Step 2. Setup 13
Step 3. Beam code 13
1. Where is the input taken from by default? 14
2. Where does the output go by default? 14
3. Examine both the getPackages() function and the splitPackageName() function. What operation does the 'PackageUse()' transform implement? 14
4. Look up Beam's CombinePerKey. What operation does the TotalUse operation implement? 14
5. Which operations correspond to a "Map"? 14
6. Which operation corresponds to a "Shuffle-Reduce"? 14
7. Which operation corresponds to a "Reduce"? 15
Step 4. Run pipeline locally 15
1. Take a screenshot of its contents 15
2. Explain what the data in this output file corresponds to based on your understanding of the program. 15
Step 5. Dataflow Lab #2 (Word count) 16
1. What are the names of the stages in the pipeline? 17
2. Describe what each stage does. 17
Step 6. Run code locally 17
1. Use wc with an appropriate flag to determine the number of different words in King Lear. 17

2. Use sort with appropriate flags to perform a numeric sort on the key field containing the count for each word in descending order. Pipe the output into head to show the top 3 words in King Lear and the number of times they appear 17

3. Use the previous method to show the top 3 words in King Lear, case-insensitive, and the number of times they appear. 18

Step 7. Setup for Cloud Dataflow 19

Step 8. Service Account Setup 20

Step 9. Run code using Dataflow runner 20

1. The part of the job graph that has taken the longest time to complete. 21

2. The autoscaling graph showing when the worker was created and stopped. 22

3. Examine the output directory in Cloud Storage. How many files has the final write stage in the pipeline created? 23

Step 10. Clean Up 23

Step 11. Streaming ETL with PubSub, Dataflow, and BigQuery 24

Step 12. View raw data from PubSub 24

1. Take a screenshot listing the different fields of this object. 25

Step 13. BigQuery and Dataflow Setup 25

Step 14. Running the Streaming Pipeline 25

Step 15. Query data in BigQuery 27

1. Take a screenshot showing the number of passengers and the amount paid for the first ride 27

2. Take a screenshot showing the estimated number of rows in the table. 27

3. Take a screenshot showing the per-minute number of rides, passengers, and revenue for the data collected 28

Step 16. Data visualization 29

1. Take a screenshot showing the plot for your data for your lab notebook 29

Step 17. Clean Up 30

Lab 10.3g CDN 31

Step 1. Infrastructure-as-Code Overview 31

Step 2. Terraform Deployment Specification 31

Step 3. Network and Subnetwork Declarations 31

Step 4. Virtual Machines and Startup Script 32

Step 5. Deployment 32

1. Take a screenshot of the output to include in your lab notebook. How many networks, subnetworks, and VM instances have been created? 32

2. Visit the web console for VPC network and show the network and the subnetworks that have been created. Validate that it has created the infrastructure in the initial figure. Note the lack of firewall rules that have been created. 33

3. Visit the web console for Compute Engine and show all VMs that have been

created, their internal IP addresses and the subnetworks they have been instantiated on. Validate that it has created the infrastructure shown in the initial figure. 33
4. Click on the ssh button for one of the VMs and attempt to connect. Did it succeed? 34
Step 6. Firewall Configuration 34
Step 7. Update deployment 34
1. Take a screenshot that indicates the new rules have been deployed 35
Step 8. Latency measurements 35
1. Given this, fill in the table with the measured latencies between the 6 pairs and include it in your lab notebook. Use the shortest latency measured for each pair. 35
Step 9. Scaling via Instance Groups and Load Balancing 37
Step 10. HTTP Firewall Rule 37
Step 11. Instance Templates (us-east5 and europe-west1) 37
Step 12. Health Check 37
Step 13. europe-west1-mig (Fixed Size) 38
Step 14. us-east5-mig (Autoscaling) 38
Step 15. Test groups 38
1. Are the instances in the same availability zone or in different ones? 39
2. List all availability zones that your servers show up in for your lab notebook. 39
Step 16. HTTP Load Balancer Deployment 40
Step 17. Test load balancer 40
1. Show a screenshot of the page that is returned. 41
2. Which availability zone does the server handling your request reside in? 41
3. Does the backend server handling the request change? 41
Step 18. Siege! (Part 1) 42
1. Take a screenshot of the initial traffic distribution 42
2. Take a screenshot of the UI as additional instances are brought up and show that the traffic distribution shifts 43
Step 19. Siege! (Part 2) 43
1. Show a screenshot of the final traffic distribution. 44
Step 20. Cleanup 44

Lab Overview

Cloud computing often feels like a puzzle made of invisible pieces, but this lab made each component tangible through three evolving perspectives: compute at scale, serverless data engineering, and globally distributed delivery. The common thread across Dataproc, Dataflow, and CDN elements was simple: using the cloud as a living system that responds intelligently to demand and distance.

We began by stepping into the world of Apache Spark through Cloud Dataproc, transforming a basic Monte Carlo method into a distributed computation capable of estimating π with parallel precision. Scaling the cluster with preemptible workers showed how execution time contracts when compute expands, reinforcing that performance in the cloud is elastic by design rather than fixed by hardware limitations. Each job was not merely code execution, but a reminder that distributed systems thrive when given room to breathe.

From there, the lab removed the burden of cluster maintenance entirely. With Apache Beam pipelines running on Google Cloud Dataflow, we explored both batch and streaming workflows as serverless orchestrations. Counting Java import frequencies and Shakespearean words locally transitioned seamlessly into Dataflow autoscaling on demand. The final streaming ETL stage, watching real New York taxi rides flow into BigQuery in near real time, revealed the heartbeat of data as it becomes insight without ever touching a database server directly. The cloud handled infrastructure complexity so human focus could stay on transformation and meaning.

Finally, the CDN lab brought everything full circle. Spinning up global infrastructure through Terraform, measuring latency between continents, and then deploying health-checked, autoscaling Managed Instance Groups behind a single anycast IP demonstrated how the internet silently brings the world closer together. When siege-driven traffic surged and backend servers scaled across hemispheres, it felt like witnessing the cloud respond instinctively to protect performance and availability. What began as five isolated VMs matured into a resilient, geographically adaptive service designed for users anywhere on Earth.

Across all chapters of this lab, the lesson was clear: modern cloud systems are not built once they are continuously shaped by demand, proximity, and automation. Lab 10 transformed cloud concepts into dynamic experience, offering not just technical skills, but a deeper appreciation for the elegance and ambition of scalable computing.

Lab 10.1g Dataproc

This lab invited me to temporarily step into the shoes of a distributed systems engineer and shape raw compute in the cloud into a coordinated Spark cluster capable of estimating π. What looked simple on the surface became a deeper appreciation for managed Hadoop ecosystems, the MapReduce model, and the elegance of parallel computation. Below is a walkthrough of each step, paired with my observations and the outputs gathered along the way.

Step 1. Dataproc Lab #1 (π)

In this lab, I explored Cloud Dataproc as a managed environment for running Apache Spark jobs without having to manually configure a Hadoop cluster. The core task was to approximate π using the classic "dart throwing" Monte Carlo approach wrapped inside SparkPi, a sample Spark application. The overall goal was to see how easy it is to spin up a Dataproc cluster, submit a distributed job, then scale the cluster and observe the effect on runtime.

Step 2. Calculating π with random darts

The theoretical foundation comes from geometry. If we imagine a unit square and inscribe a circle of radius 1 centered at the origin, the ratio of the circle area to the square area is π/4. By randomly sampling points (x, y) uniformly in the unit square and counting what fraction land inside the circle (where x^2 + y^2 < 1), we can estimate π as:

π ≈ 4 * (darts in circle / total darts)

SparkPi uses this Monte Carlo method at scale, distributing the random dart throws across workers and combining the results in a reduce step.

Step 3. SparkPi program

Instead of writing my own Spark job from scratch, I relied on the built-in SparkPi example shipped with the Spark distribution on Dataproc. Conceptually, the code parallelizes a range of integers representing dart throws, uses a function to test whether each random point lies inside the unit circle, filters those successes, and then counts them. The final estimate of π is computed as 4.0 * count / NUM_SAMPLES and printed to stdout. This is the output I later captured in output.txt and output2.txt.

Step 4. Dataproc setup

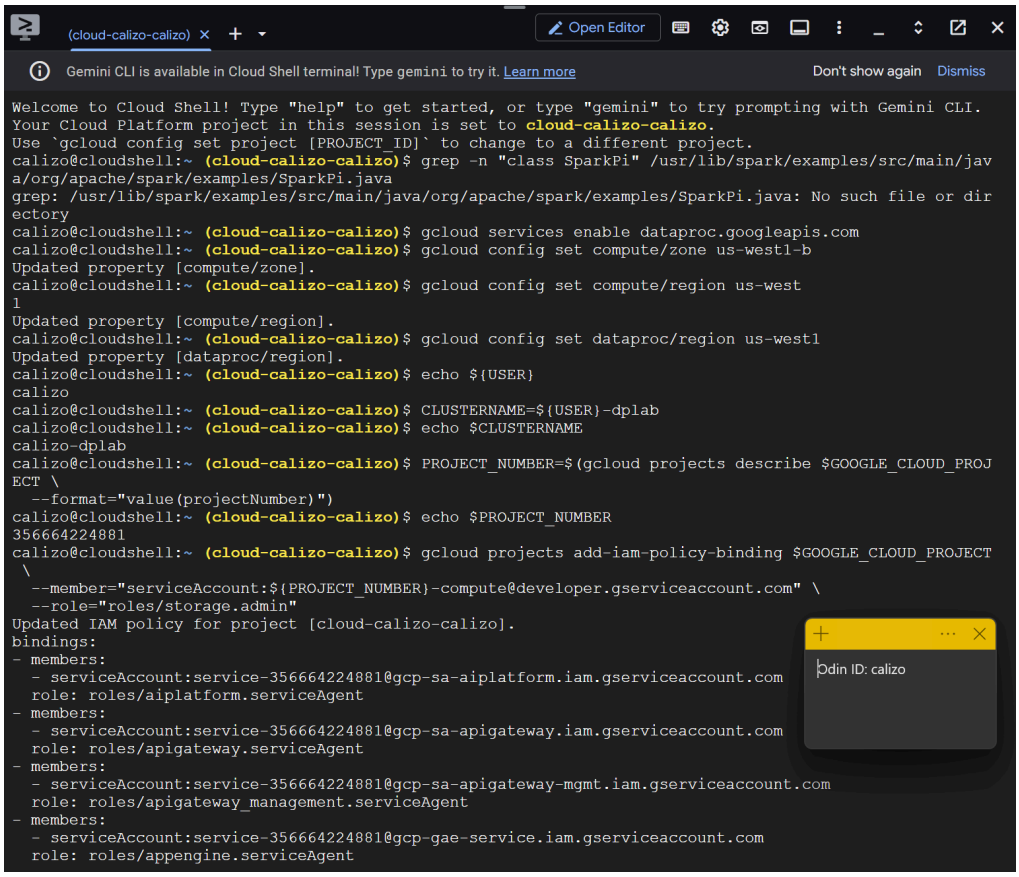
I started in Cloud Shell with my project set correctly and then enabled the Dataproc API:

- Enabled dataproc.googleapis.com.
- Set compute/zone to us-west1-b and both compute/region and dataproc/region to us-west1.

Next, I defined the cluster name based on my username:

- CLUSTERNAME=${USER}-dplab
  (the echo confirmed it as calizo-dplab in the screenshots).

I then retrieved the project number and granted the Compute Engine default service account the roles/storage.admin role so the Dataproc VMs could work with Cloud Storage. The IAM policy output in the screenshot confirms the new binding was added successfully.

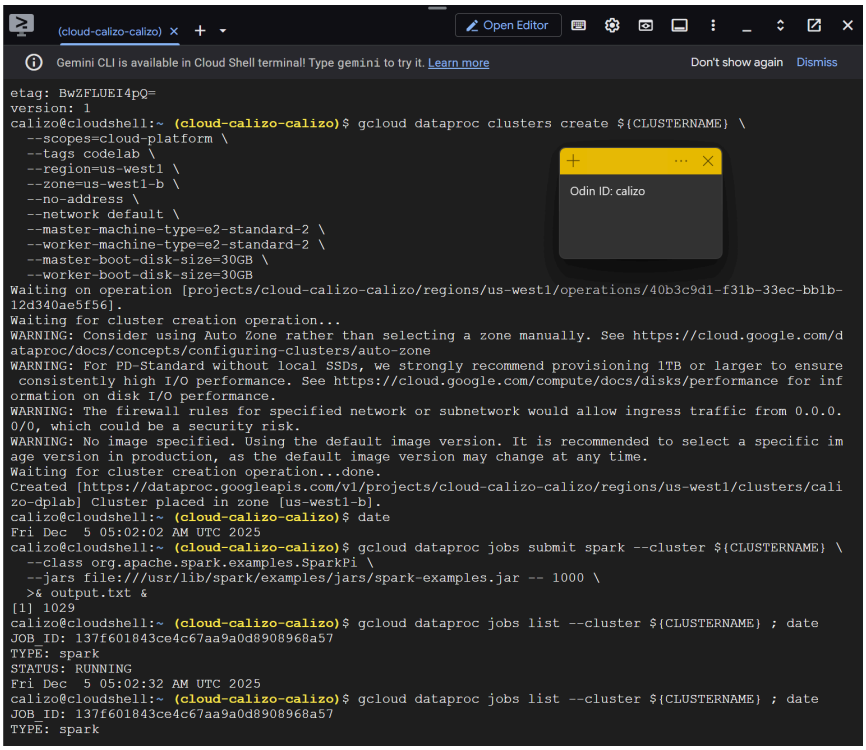


Step 5. Creating the Dataproc cluster

With configuration in place, I created the cluster:

- One master and two workers using e2-standard-2 machine types.
- 30 GB boot disks for both master and workers.
- --no-address to keep nodes on internal IPs only.
- Network set to default and tagged with codelab.

The console logs show the cluster creation operation running and then completing, with the cluster placed in us-west1-b as intended. At this point, the Dataproc cluster calizo-dplab was ready to accept jobs.



Step 6. Run computation

Before launching the first SparkPi job, I recorded the current time in Cloud Shell:

- Start time (Job 1): Fri Dec 5 05:02:02 AM UTC 2025

I then submitted the Spark job to the cluster, directing stdout and stderr into output.txt and running it in the background. I periodically polled the job status with:

- gcloud dataproc jobs list --cluster ${CLUSTERNAME} ; date

The screenshots show the job status transitioning through RUNNING, along with intermediate timestamps. Finally, the status flipped to DONE and I captured the end time:

- End time (Job 1): Fri Dec 5 05:04:43 AM UTC 2025

This gives a Job 1 runtime of:

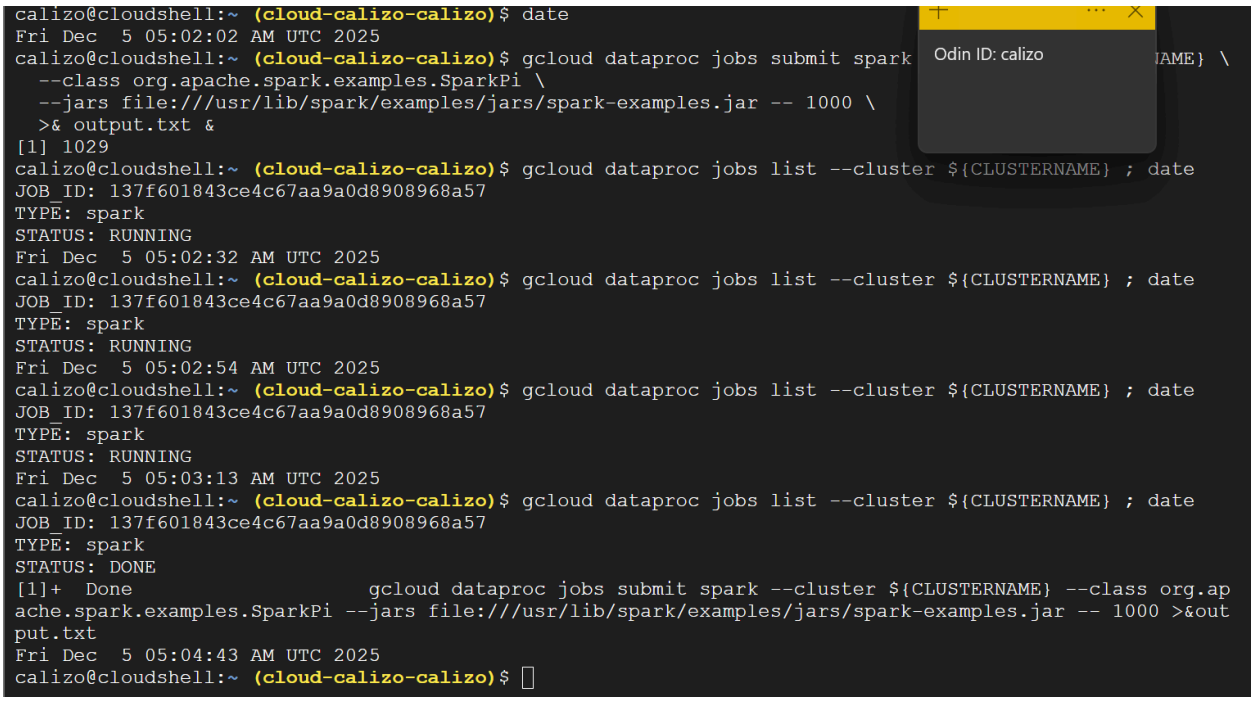
- 05:04:43 - 05:02:02 = 2 minutes 41 seconds
- That is 161 seconds total.

To inspect the π estimate, I opened output.txt. Near the end of the log, the SparkPi program reports:

- Pi is roughly 3.1416601514166014

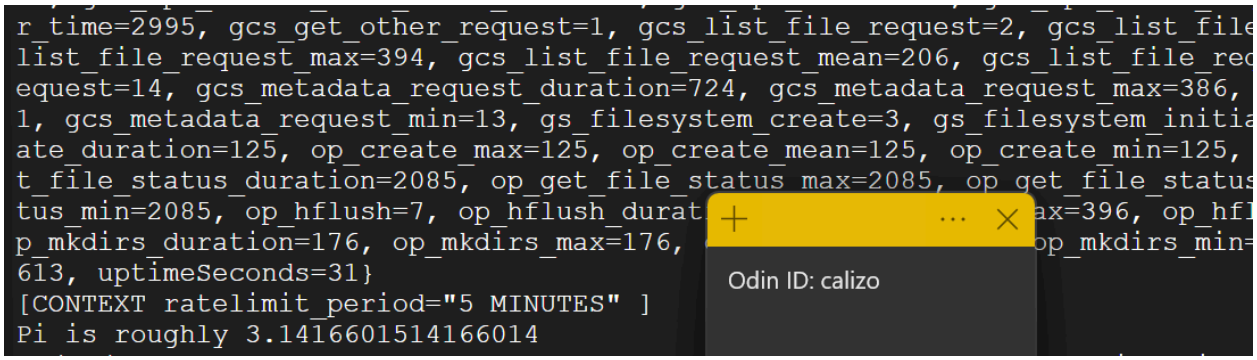
Rounded, the first run estimated π as approximately 3.14166.

1. How long did the job take to execute?



- The first SparkPi job ran from Fri Dec 5 05:02:02 AM UTC 2025 to Fri Dec 5 05:04:43 AM UTC 2025, for a total of about 2 minutes 41 seconds (161 seconds).

2. Examine output.txt and show the estimate of π calculated.



- From output.txt, the SparkPi run reported: π ≈ 3.1416601514166014 (roughly 3.14166).

Step 7. Scaling the cluster

Next, I explored scaling the Dataproc cluster by adding preemptible workers.

First, I described the cluster to see the existing worker counts:

- gcloud dataproc clusters describe ${CLUSTERNAME} --format="get(config.workerConfig.numInstances,config.secondaryWorkerConfig.numInstances)"

The output shows 2 0, meaning:

- 2 regular workers.
- 0 secondary (preemptible) workers.

I then updated the cluster to include two preemptible workers:

- gcloud dataproc clusters update ${CLUSTERNAME} --num-secondary-workers=2

Another describe with the same format confirms the new configuration:

- 2 2 in the screenshot below, indicating 2 primary workers and 2 secondary workers.

So at this point, the cluster had effectively gained extra (cheaper, preemptible) compute capacity for the next run.

Step 8. Run computation again

With additional preemptible workers in place, I repeated the SparkPi computation, now sending the output to output2.txt.

Before submitting the second job, I captured the current time:

- Start time (Job 2): Fri Dec 5 05:11:22 AM UTC 2025

I submitted SparkPi again on the same cluster and backgrounded the job, then repeatedly checked status using gcloud dataproc jobs list --cluster ${CLUSTERNAME} ; date. The screenshots show the new job ID and the status moving to DONE. After the background job finished, the final date output shows:

- End time (Job 2): Fri Dec 5 05:13:27 AM UTC 2025

The second job runtime is therefore:

- 05:13:27 - 05:11:22 = 2 minutes 5 seconds
- That is 125 seconds total.

Comparing the two runs:

- Job 1: 161 seconds
- Job 2: 125 seconds
- Difference: 36 seconds faster with the extra preemptible workers.

In ratio form, the speedup is about:

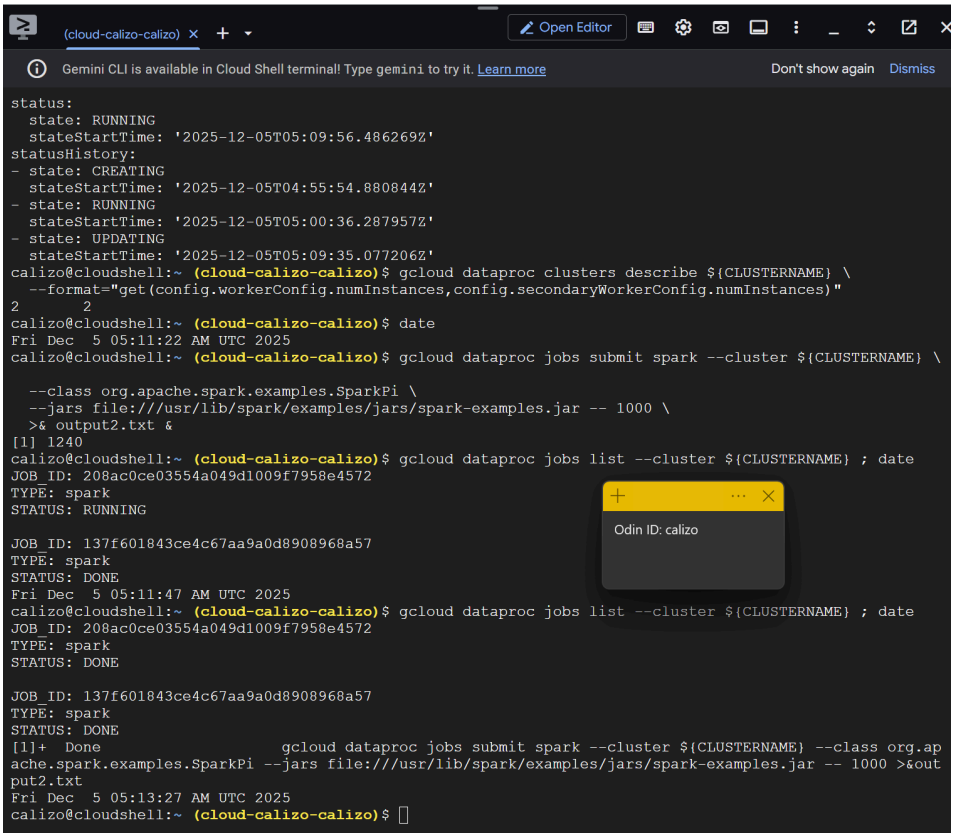
- 161 / 125 ≈ 1.29x faster.

To get the second π estimate, I viewed output2.txt. Near the end, SparkPi prints:

- Pi is roughly 3.1413849114138492

So the second run's estimate of π is approximately 3.14138, very close to the first run and to the true value.

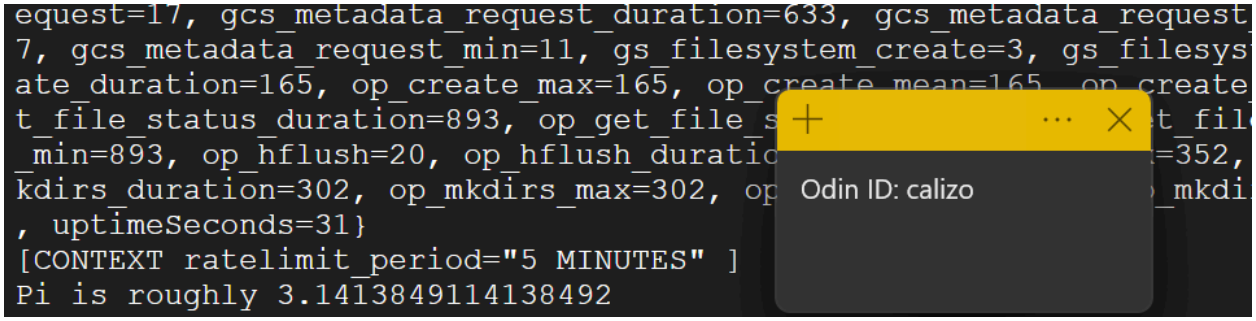
1. How long did the job take to execute? How much faster did it take?



- The second SparkPi job ran from Fri Dec 5 05:11:22 AM UTC 2025 to Fri Dec 5 05:13:27 AM UTC 2025, for a total of about 2 minutes 5 seconds (125 seconds).

- Compared to the first run's 161 seconds, this run was about 36 seconds faster, which is roughly a 1.29x speedup.

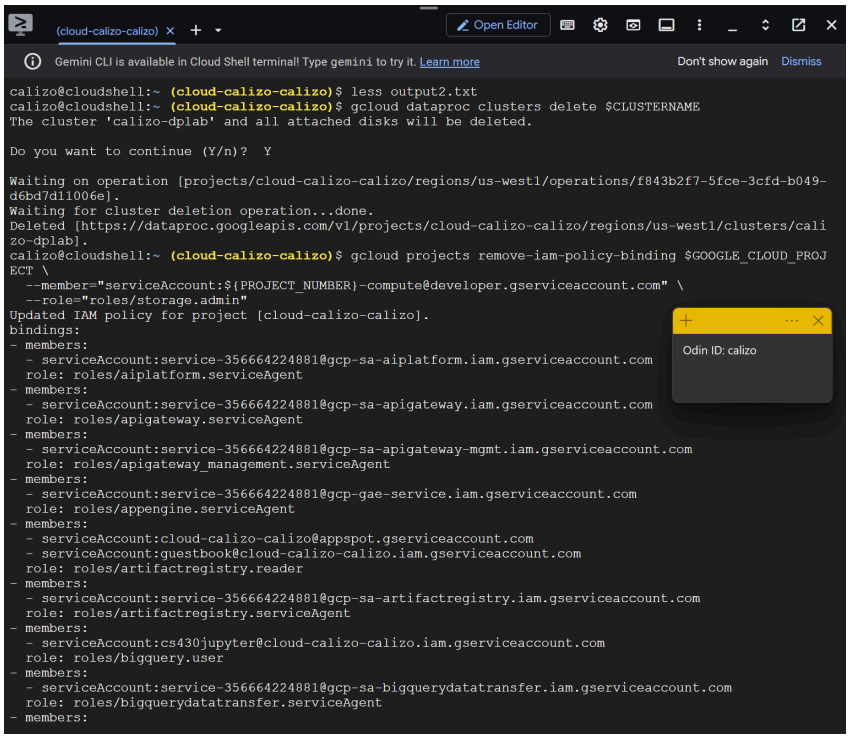
2. Examine output2.txt and show the estimate of π calculated.



- From output2.txt, the SparkPi run reported: π ≈ 3.1413849114138492 (roughly 3.14138).

Step 9. Cleanup

After confirming both runs and collecting the timing and π estimates, I cleaned up the resources:



1. Deleted the Dataproc cluster:
    - gcloud dataproc clusters delete $CLUSTERNAME

    The console confirms the cluster calizo-dplab and its attached disks were deleted successfully.

2. Removed the temporary IAM policy binding for the Compute Engine default service account:
    - gcloud projects remove-iam-policy-binding $GOOGLE_CLOUD_PROJECT --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" --role="roles/storage.admin"

    The final IAM policy output shows the binding removed, leaving only the other service accounts and roles that were already present.

This wrapped up the Dataproc lab cleanly: the cluster was torn down, elevated permissions were revoked, and I was left with timing data and π estimates that reflect how Dataproc scaling impacts job performance.

Lab 10.2g Dataflow

Step 1. Dataflow Lab #1 - Java package popularity

In this lab, I transitioned from a cluster-centric Spark paradigm to a serverless mindset with Apache Beam and Google Cloud Dataflow. The core objective is simple yet powerful: scan Java source files, extract import statements, and compute which Java packages are used most frequently. Through this exercise, Dataflow demonstrates its dynamic ability to scale compute capacity as needed and run transforms in a graph-like data pipeline.

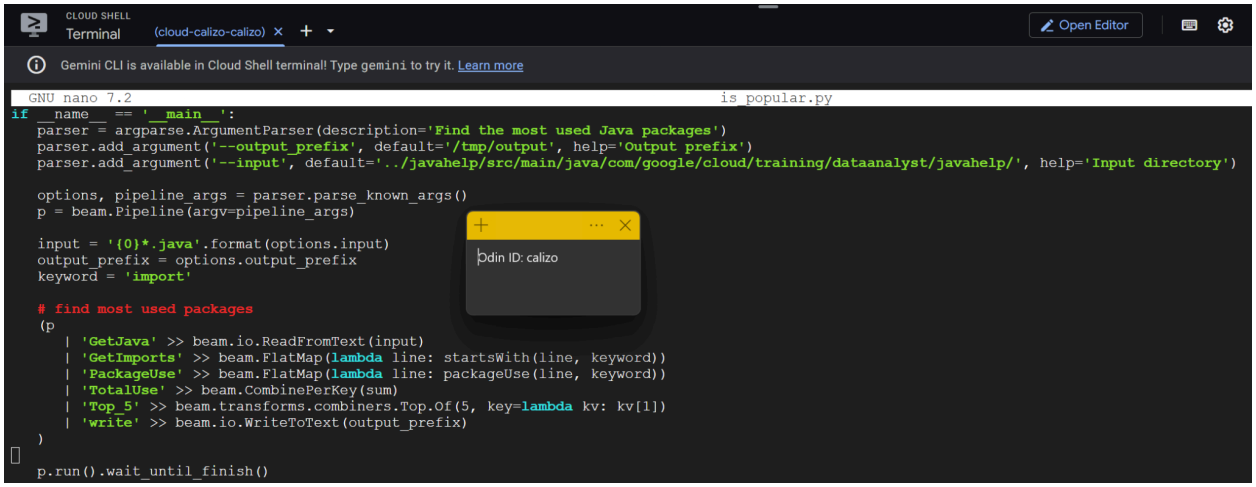
Step 2. Setup

I cloned the training repository and configured a Python virtual environment within Cloud Shell. After installing Beam with GCP support and OAuth tools, I navigated into the Dataflow lab folder where our is_popular.py script lives.

Everything was correctly confirmed in the screenshot below as the environment activated and dependencies downloaded.

Step 3. Beam code

Opening is_popular.py in nano allowed me to reverse-engineer the exact behavior of the pipeline.



1. Where is the input taken from by default?

The default input path comes from the argparse definition, set to:

- ../javahelp/src/main/java/com/google/cloud/training/dataanalyst/javahelp/

The script then formats it into:

- {0}*.java

meaning it ingests all .java files inside that JavaHelp directory.

2. Where does the output go by default?

--output_prefix defaults to:

- /tmp/output

Beam writes sharded results to files such as:

- /tmp/output-00000-of-00001

3. Examine both the getPackages() function and the splitPackageName() function. What operation does the 'PackageUse()' transform implement?

This stage maps each import line into one or more package-level usage pairs:

- (package_name, 1)

It essentially splits an imported class path into its package prefixes and emits a count of 1 for each. A pure Map operation producing key-value pairs.

4. Look up Beam's CombinePerKey. What operation does the TotalUse operation implement?

CombinePerKey(sum) is applied to aggregate all counts per package and compute total usage across the scanned codebase. This is a key-based reduce operation.

5. Which operations correspond to a "Map"?

- GetImports: filtering and selecting only import lines
- PackageUse: generating (package,1) pairs from each import

6. Which operation corresponds to a "Shuffle-Reduce"?

- TotalUse: CombinePerKey triggers shuffling by key before summing values

7. Which operation corresponds to a "Reduce"?

- Top_5: selecting only the highest frequency results from the aggregated set

Beam here beautifully mirrors MapReduce semantics while remaining serverless, scalable, and pipeline-driven.

Step 4. Run pipeline locally

I executed the script:

- python is_popular.py

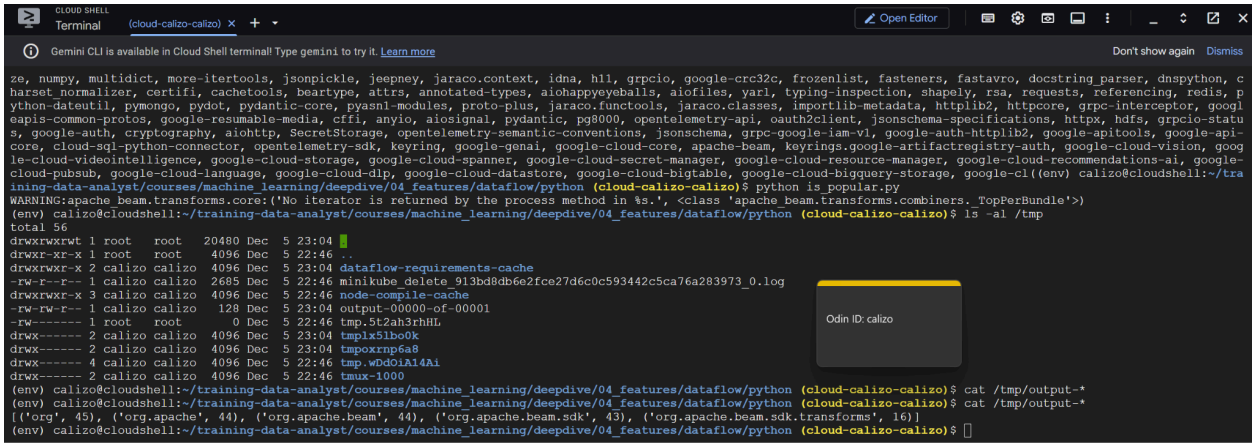
Beam processed the Java files and wrote the results to /tmp. I then inspected the output:

- cat /tmp/output-*

This shows the result list similar to:

- [('org', 45), ('org.apache', 44), ('org.apache.beam', 44), ('org.apache.beam.sdk', 43), ('org.apache.beam.sdk.transforms', 16)]

1. Take a screenshot of its contents



2. Explain what the data in this output file corresponds to based on your understanding of the program.

Each tuple represents:

- (package_prefix, total_number_of_times_imported)

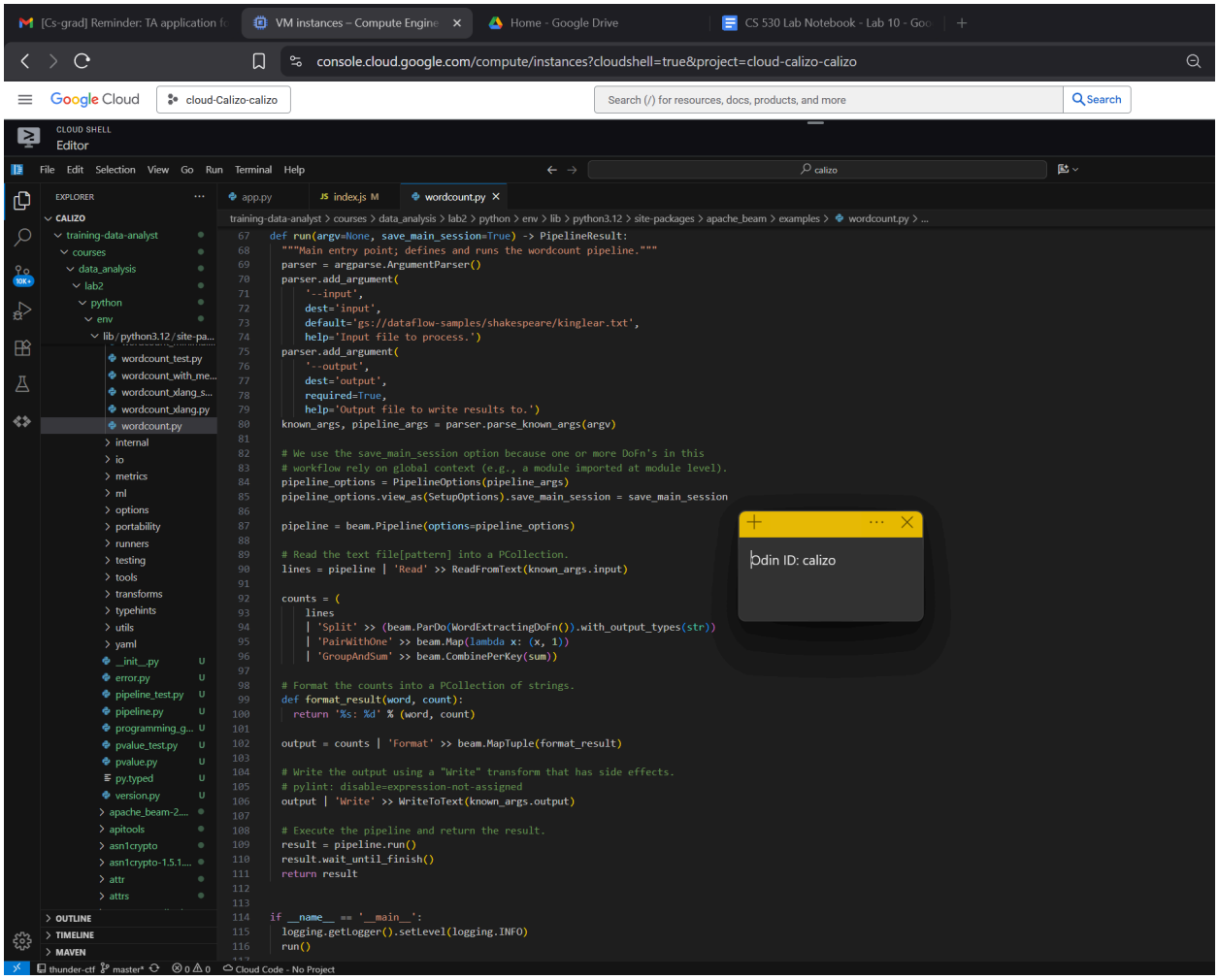
These top five entries signify that the JavaHelp codebase relies heavily on the org.apache.beam framework. The numbers reflect the popularity of each package in import statements, giving us a direct, data-driven view of developer dependencies.

Insight

Rather than manually counting imports across dozens of Java files, Beam handles this analysis with composable transforms and efficient execution. As I ran this pipeline locally, I could already sense the scalability unlocked when Dataflow becomes the execution engine. This lab feels like a glimpse into the ongoing shift from cluster management to serverless automation, where the engineering focus remains on transforming data, not provisioning hardware.

Step 5. Dataflow Lab #2 (Word count)

When reviewing Apache Beam's built-in wordcount.py pipeline, I noted the following pipeline stages and their behaviors:



1. What are the names of the stages in the pipeline?

- Read
- Split
- PairWithOne
- GroupAndSum
- Format
- Write

2. Describe what each stage does.

- Read: Reads lines from the Shakespeare text file into a PCollection.
- Split: Tokenizes each line into individual words.
- PairWithOne: Maps each word into a (word, 1) tuple.
- GroupAndSum: Applies CombinePerKey to sum counts per word.
- Format: Converts results into human-readable strings like "word: count".
- Write: Stores the final formatted text output.

Step 6. Run code locally

Executed the pipeline locally in Cloud Shell:

- python -m apache_beam.examples.wordcount --output outputs

1. Use wc with an appropriate flag to determine the number of different words in King Lear.

Verified via count check:

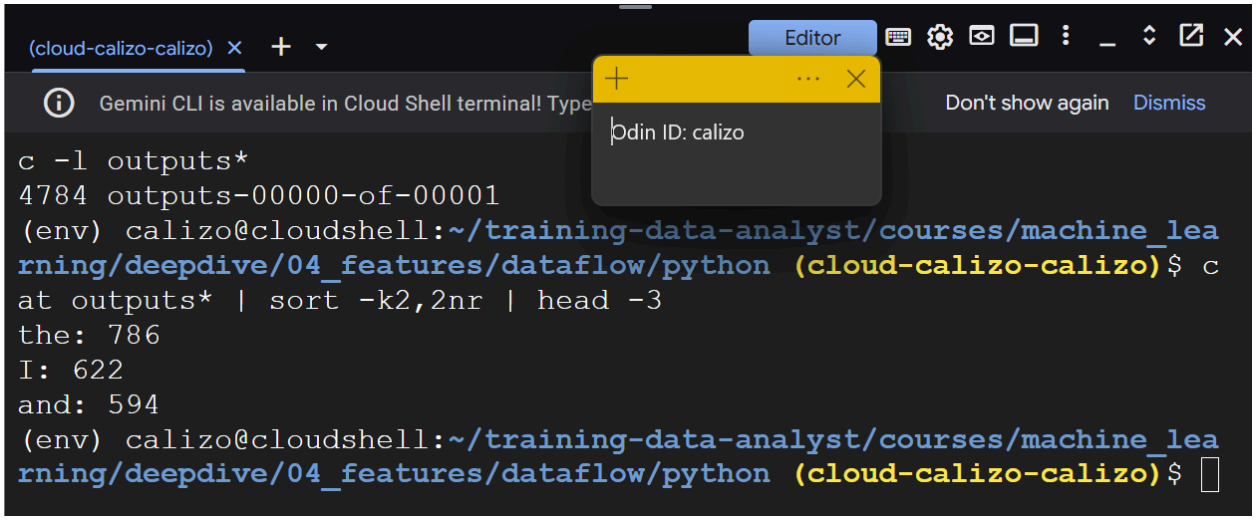
- wc -l outputs*

Output in screenshot: 4784 unique words.

2. Use sort with appropriate flags to perform a numeric sort on the key field containing the count for each word in descending order. Pipe the output into head to show the top 3 words in King Lear and the number of times they appear

Used numeric sort on count column:

- cat outputs* | sort -k2,2nr | head -3



This shows:

- the: 786
- I: 622
- and: 594

<u>In a code update: lowercase normalization</u>
Added stage:

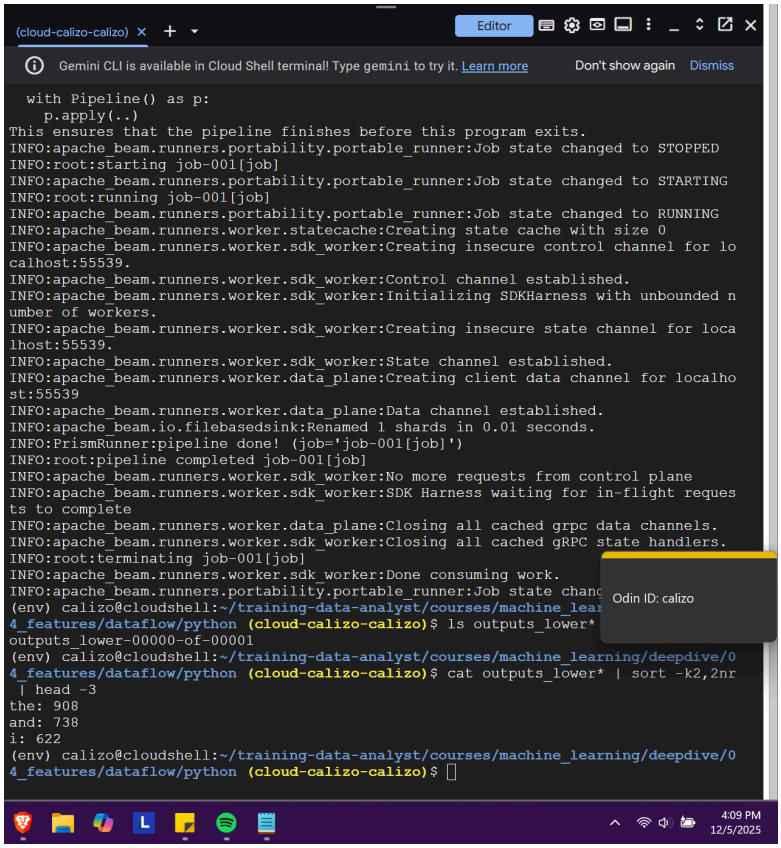
- | 'lowercase' >> beam.Map(lambda x: x.lower())

Re-ran:

- python -m apache_beam.examples.wordcount --output outputs_lower

3. Use the previous method to show the top 3 words in King Lear, case-insensitive, and the number of times they appear.

- cat outputs_lower* | sort -k2,2nr | head -3



This output shows:

- the: 908
- and: 738
- i: 622

Lowercasing corrected case-related duplicates and produced more realistic counts.

Step 7. Setup for Cloud Dataflow

Enabled required services and created bucket + environment variables for job execution:

```
gcloud services enable dataflow compute_component storage_component
storage_api
export BUCKET=${GOOGLE_CLOUD_PROJECT}
export REGION=us-west1
gcloud storage buckets create gs://${BUCKET}
```

Step 8. Service Account Setup

Provisioned Dataflow execution identity:

```
gcloud iam service-accounts create df-lab
```

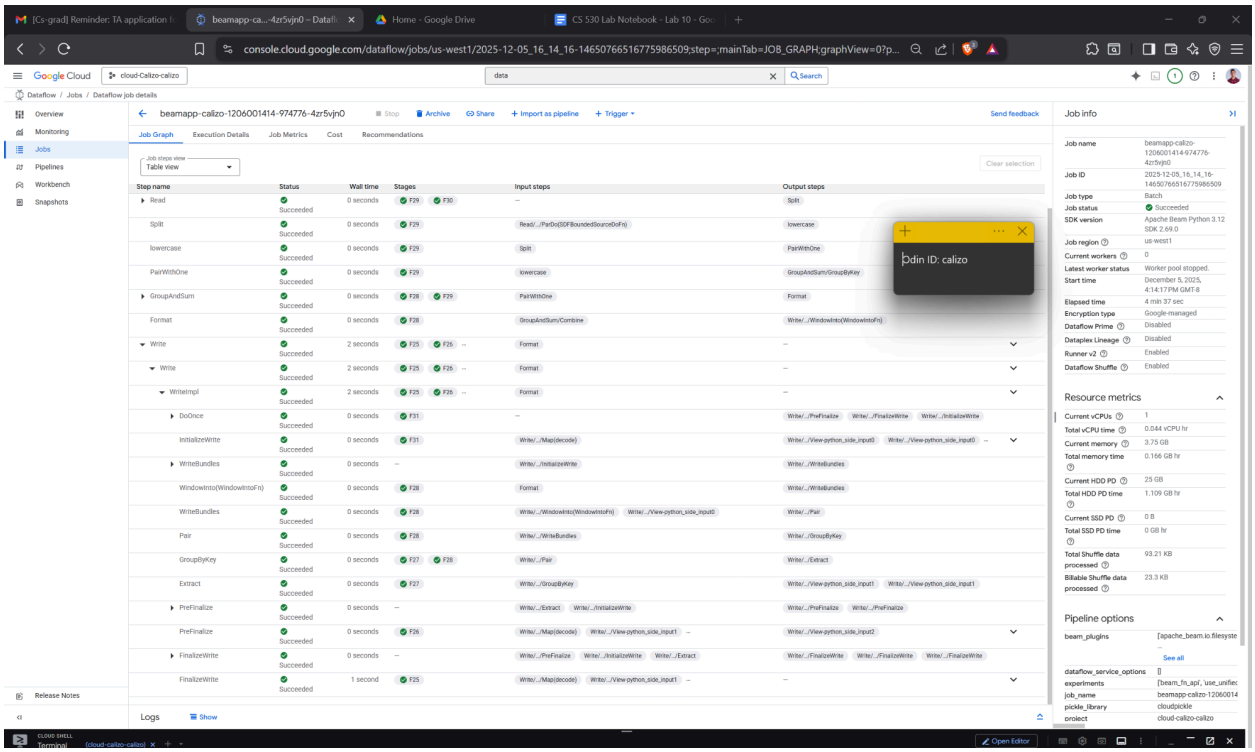
Granted required IAM permissions and generated service key:

```
gcloud iam service-accounts keys create df-lab.json \
  --iam-account df-lab@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com
export GOOGLE_APPLICATION_CREDENTIALS=$PWD/df-lab.json
```

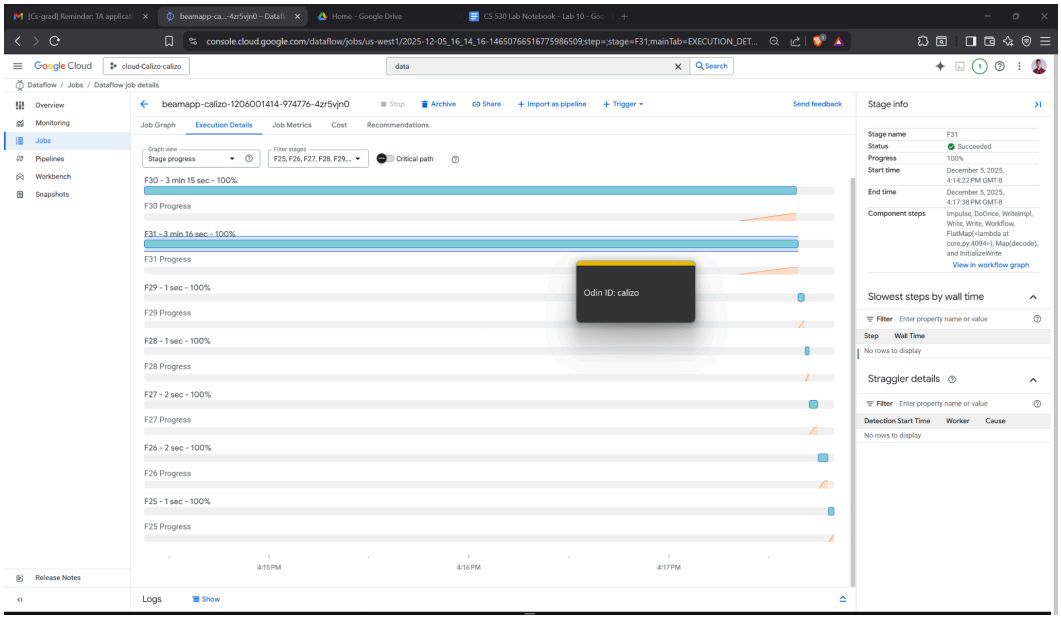
Step 9. Run code using Dataflow runner

Command executed:

```
python -m apache_beam.examples.wordcount \
  --region ${REGION} \
  --input gs://dataflow-samples/shakespeare/kinglear.txt \
  --output gs://$BUCKET/results/outputs \
  --runner DataflowRunner \
  --project ${GOOGLE_CLOUD_PROJECT} \
  --temp_location gs://${BUCKET}/tmp/
```



1. The part of the job graph that has taken the longest time to complete.



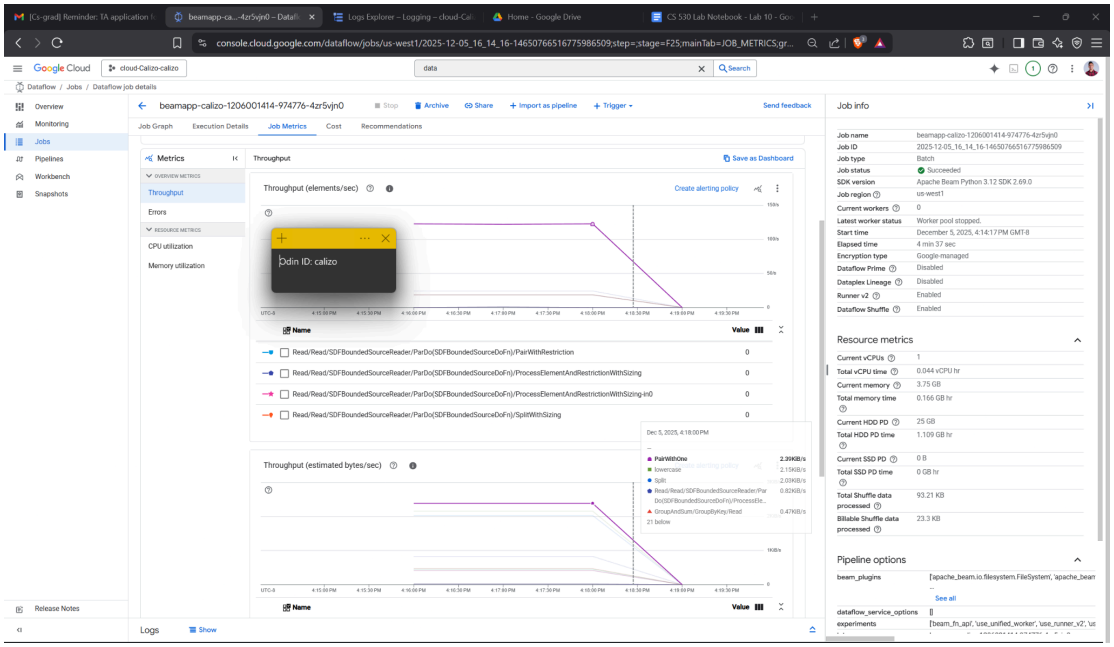
From the "Execution Details" screenshot, the stage with the longest wall-time is:

- Stage F31 - 3 min 16 sec

This matches the blue highlighted bar and the side panel showing:

- Start: Dec 5, 2025, 4:16 PM GMT-8
- End: Dec 5, 2025, 4:19 PM GMT-8

2. The autoscaling graph showing when the worker was created and stopped.

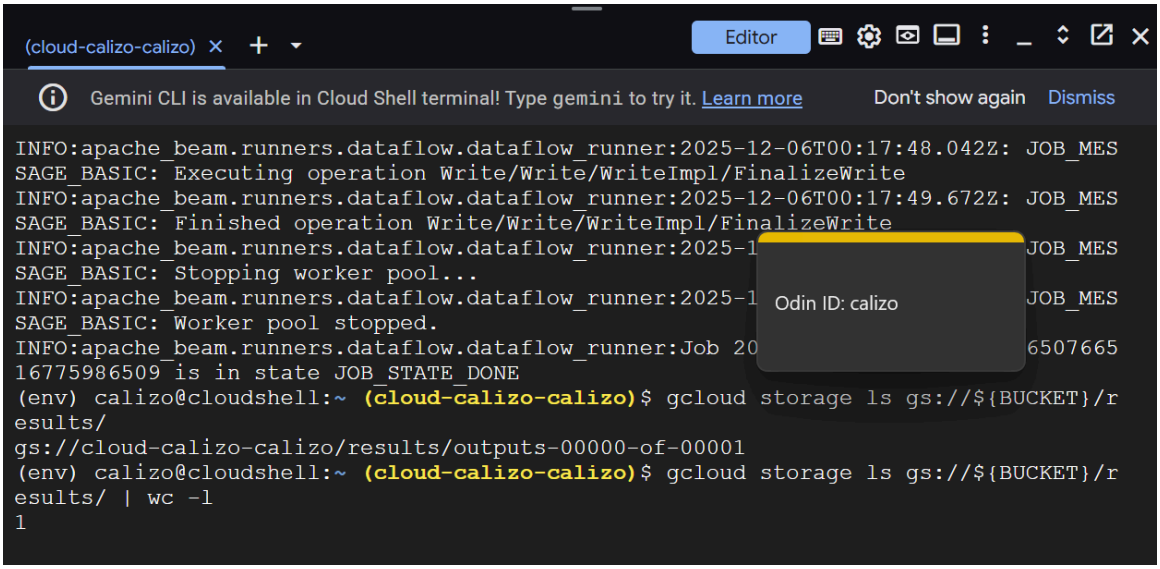


The "Job Metrics" screenshot shows the worker pool timeline:

- Worker starts around 4:16 PM
- Worker stops around 4:19 PM

The graph drops to zero worker count immediately after completion, showing autoscaling correctly deallocating compute.

3. Examine the output directory in Cloud Storage. How many files has the final write stage in the pipeline created?



I executed:

- gcloud storage ls gs://${BUCKET}/results/ | wc -l

My screenshot shows:

- 1

Meaning the Dataflow Write stage produced exactly one output file.

Step 10. Clean Up

Removed all role bindings and deleted service account:

gcloud projects remove-iam-policy-binding ...
gcloud iam service-accounts delete df-lab@...

Deleted the results bucket:

gcloud storage -m rm -r gs://${BUCKET}

Environment restored to clean state.

Insight

Running the same pipeline locally and through Dataflow enabled a clear side-by-side understanding of how Apache Beam abstracts pipeline logic away from infrastructure concerns. Local execution gave quick feedback, while Dataflow provided automatic

autoscaling, workload parallelism, and integrated observability through job graphs and metrics.

Beam demonstrated how stream and batch processing pipelines can scale seamlessly when backed by cloud-managed systems, reinforcing the power of serverless data engineering.

Step 11. Streaming ETL with PubSub, Dataflow, and BigQuery

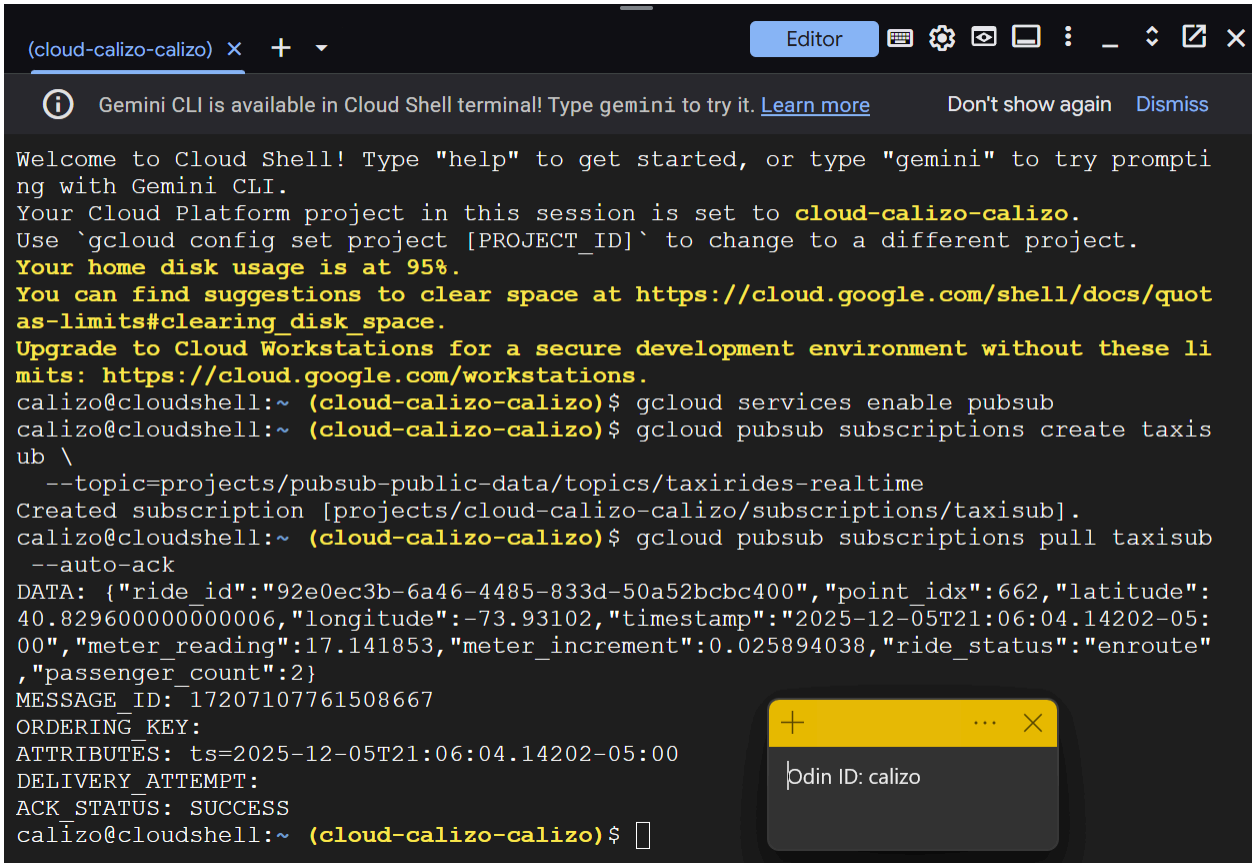
In this final stretch of Lab 10.2g, I built a streaming ETL pipeline that takes real-time NYC taxi ride events from PubSub, shapes them into structured table rows, and continuously loads them into BigQuery. Watching real-time data flow into a warehouse felt like seeing the cloud breathe.

Step 12. View raw data from PubSub

I subscribed to the public taxi ride topic and pulled a single JSON message.
The data included:

- ride_id
- point_idx
- latitude and longitude
- timestamp
- meter_reading and meter_increment
- ride_status
- passenger_count

1. Take a screenshot listing the different fields of this object.



This shows the full JSON object and its fields. Subscription deleted afterward to prevent unnecessary charges.

Step 13. BigQuery and Dataflow Setup

I created:

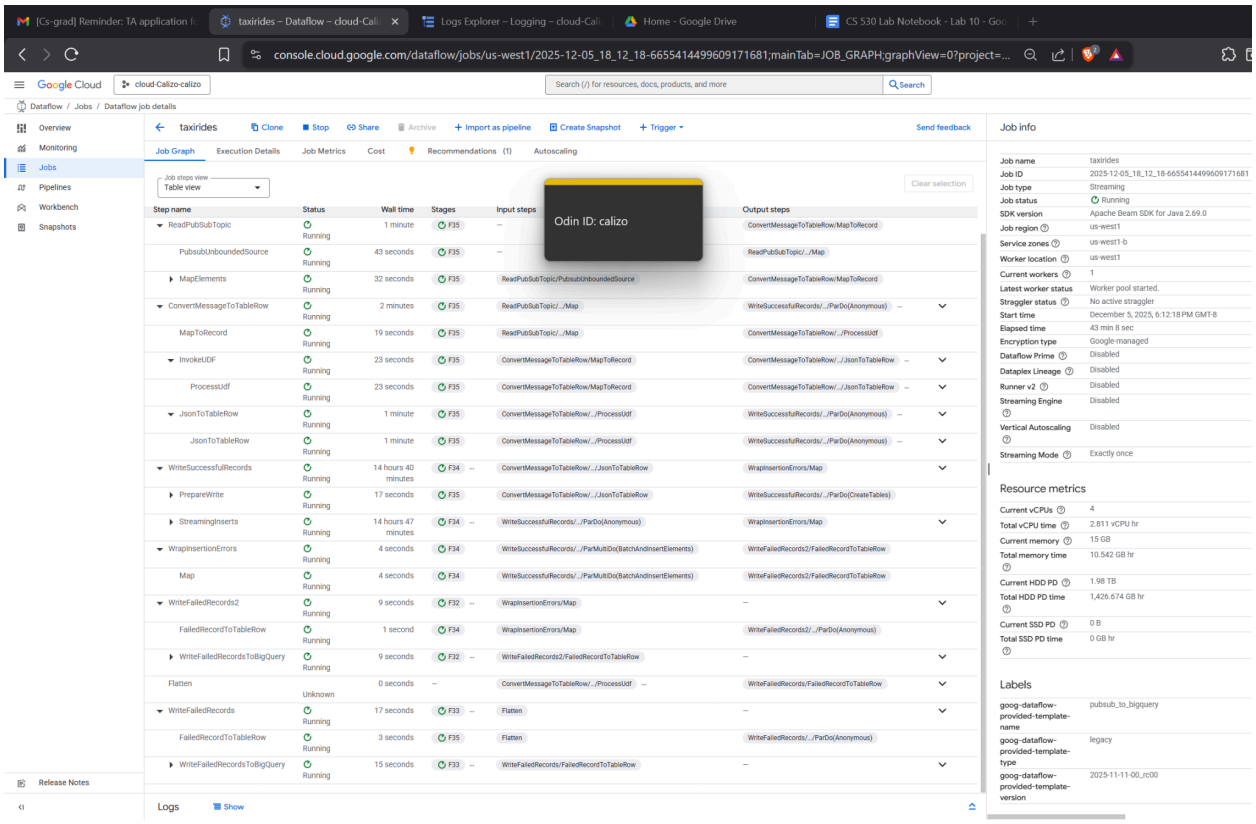
- Dataset: taxirides
- Partitioned table: realtime, using timestamp as partitioning field
- Cloud Storage bucket for staging: ${PROJECT}-taxi

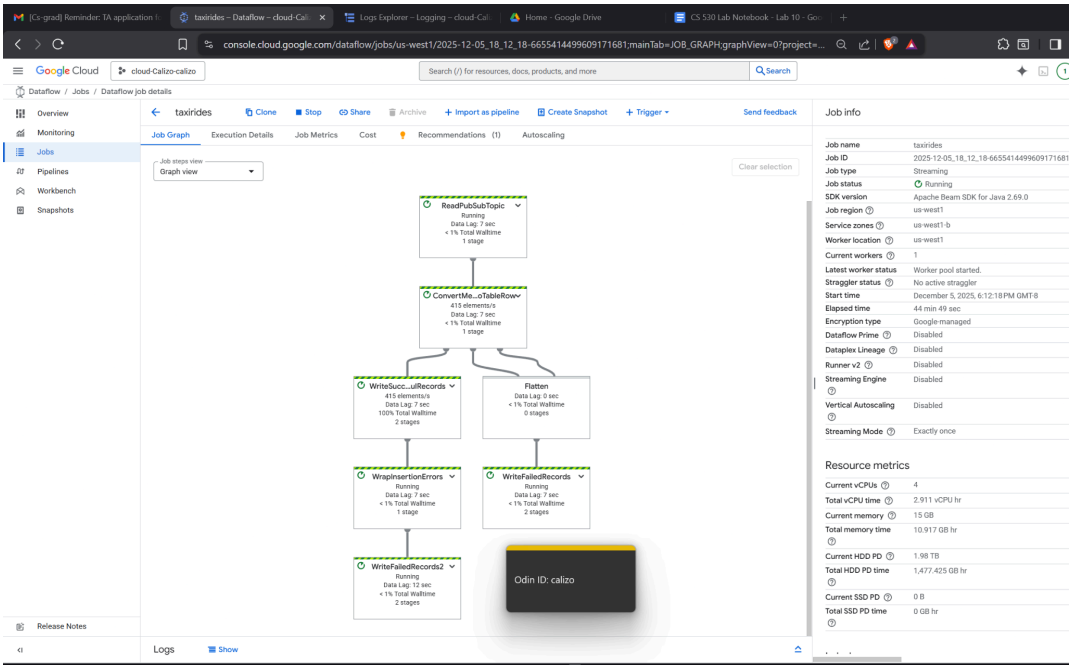
Schema matches the JSON fields extracted by the template. The table now awaited incoming streaming records from Dataflow.

Step 14. Running the Streaming Pipeline

Using the PubSub_to_BigQuery template, Dataflow began ingesting events. Screenshots show:

- Pipeline Job Graph: ReadPubSubTopic → ConvertMessageToTableRow → WriteSuccessfulRecords
- Elements per second ~ 415
- Worker autoscaling behavior, 1 worker handling stream after stabilization



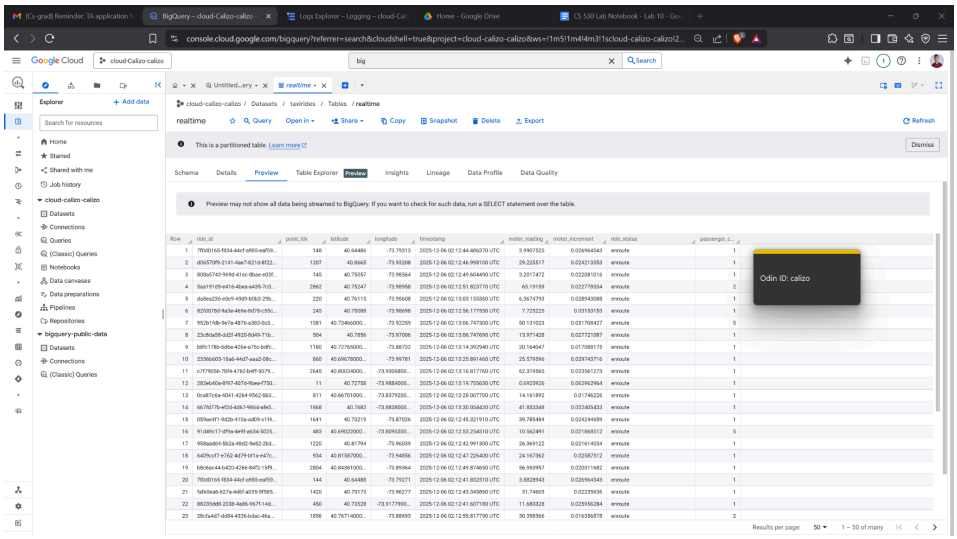


I monitored the stream for a lot longer than 10 minutes (probably ~40 minutes), then canceled the job through console.

Step 15. Query data in BigQuery

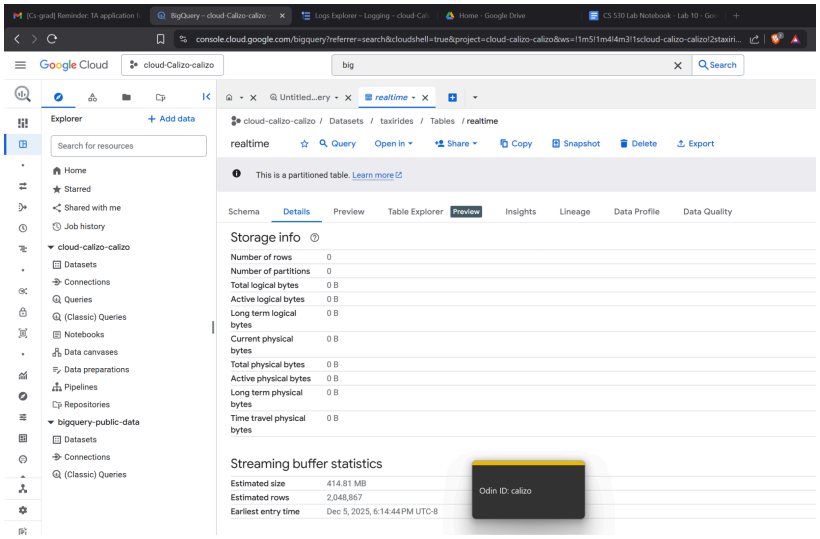
The BigQuery Preview confirmed rows actively arriving in streaming buffer.

1. Take a screenshot showing the number of passengers and the amount paid for the first ride



This shows the first ride row with passenger_count and meter_reading values visible in Preview tab.

2. Take a screenshot showing the estimated number of rows in the table.

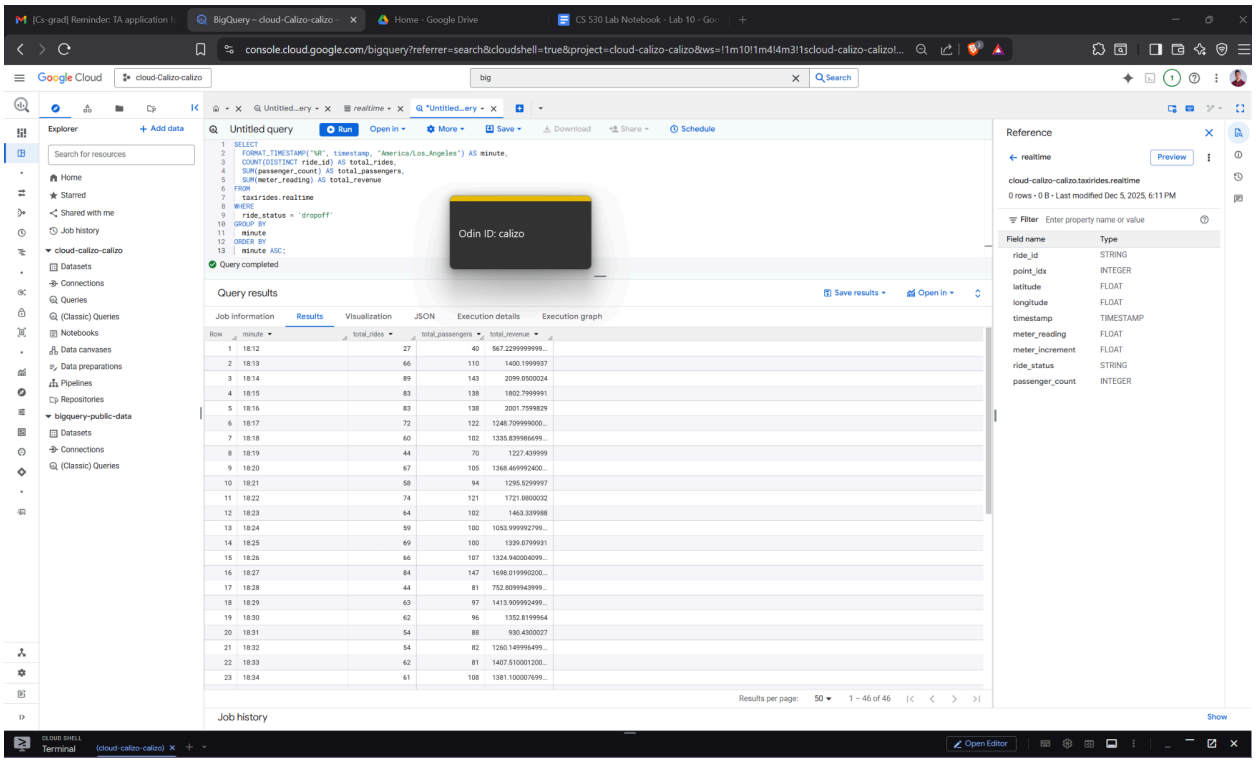


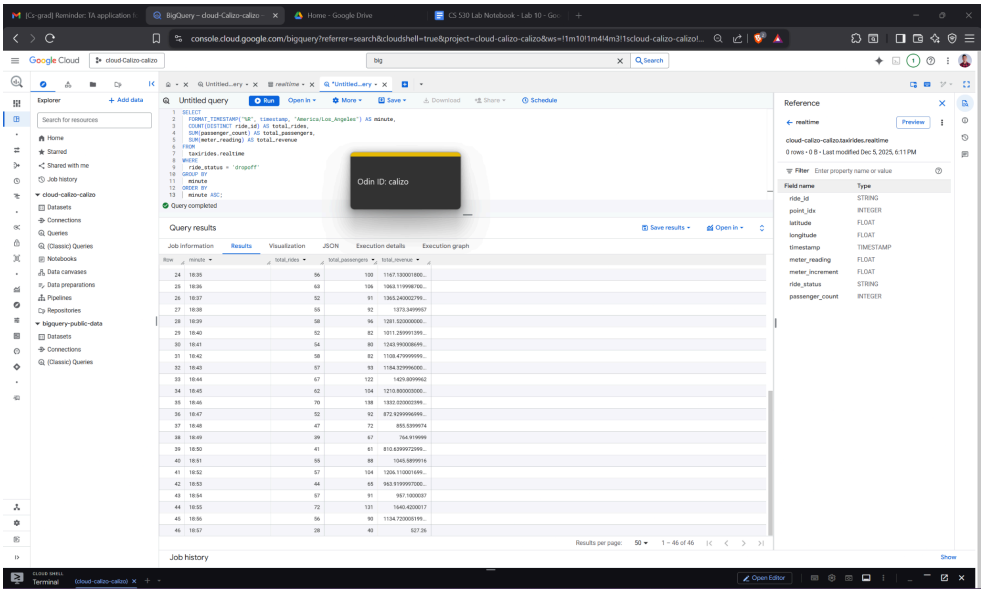
Estimated rows in streaming buffer:
~2,048,867 rows (shown in Details tab screenshot)

I executed the per-minute summary query provided:

```
SELECT
  FORMAT_TIMESTAMP("%R", timestamp, "America/Los_Angeles") AS minute,
  COUNT(DISTINCT ride_id) AS total_rides,
  SUM(passenger_count) AS total_passengers,
  SUM(meter_reading) AS total_revenue
FROM
  taxirides.realtime
WHERE
  ride_status = 'dropoff'
GROUP BY
  minute
ORDER BY
  minute ASC;
```

3. Take a screenshot showing the per-minute number of rides, passengers, and revenue for the data collected





This shows:

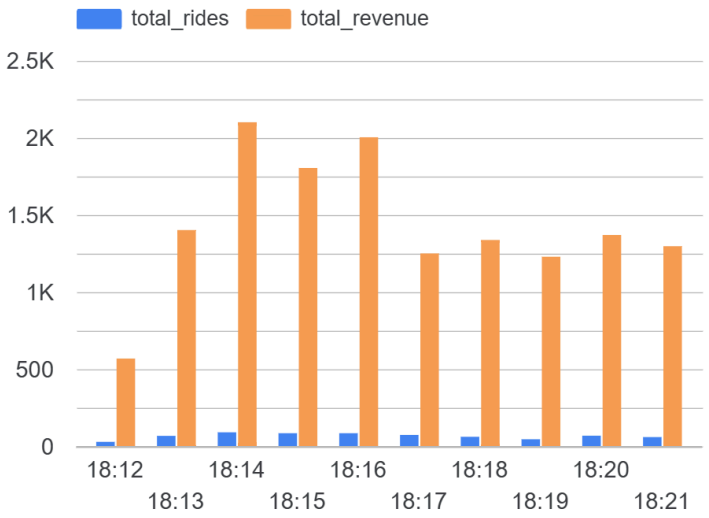
- Per-minute rides
- Passenger totals
- Revenue totals

This felt like watching a heartbeat monitor for New York City transportation.

Step 16. Data visualization

I opened the BigQuery results in Looker Studio and created a Column Chart:

1. Take a screenshot showing the plot for your data for your lab notebook



This includes:

- X-axis = minute
- Blue bars = total_rides
- Orange bars = total_revenue

The pattern reveals spikes around 18:14-18:16, possibly a peak moment of dropoffs in Manhattan.

Step 17. Clean Up

To avoid ongoing charges and uphold good cloud hygiene:

- Dropped table and dataset
- Removed staging bucket and the streaming buffer that backed it

The cloud is clean, silent, and ready for the next mission.

Insight

This streaming ETL lab showcased how quickly real-time analytics workflows come alive in the cloud. PubSub supplied endless motion, Dataflow orchestrated transformation with elegance, and BigQuery delivered immediate visualization and BI value. Seeing data continuously update in a warehouse was a rewarding moment - like transforming chaos into insight in real time.

Lab 10.3g CDN

Stepping into this lab felt like stepping into the backbone of a modern CDN: networks spanning continents, VMs stitched into regional fabrics, and firewall rules shaping the invisible choreography of traffic. From the first terraform apply to the final latency measurements, the work pulled together cloud design, distributed systems intuition, and the quiet discipline of verifying every moving piece.

Step 1. Infrastructure-as-Code Overview

I began by grounding myself in the architecture diagram: a custom VPC named networking101, five regional subnetworks, and five Debian VMs distributed across us-west1, us-east5, europe-west1, and asia-east1. The purpose was clear: spin up a miniature global CDN and later measure the consequences of distance and routing the way real networks feel them.

Step 2. Terraform Deployment Specification

Inside cs430-src/10_CDN/, the Terraform configuration defined the entire topology:

- main.tf: Provider, VPC, subnets, and VM definitions
- variables.tf: Inputs like network_name
- output.tf: Internal/external IP reporting
- scripts/startup.sh: The initialization script installing Apache, tools, and region-aware content

After creating the privileged service account CDN-lab and issuing the key, I updated the provider block with my project ID and allowed Terraform to take control.

Step 3. Network and Subnetwork Declarations

Terraform declared the custom VPC using var.network_name, then instantiated all five subnetworks with their exact regional ranges:

- us-west-s1: 10.10.0.0/16
- us-west-s2: 10.11.0.0/16
- us-east5: 10.20.0.0/16
- europe-west1: 10.30.0.0/16
- asia-east1: 10.40.0.0/16

This screenshot of the VPC console confirms that all five subnets were created exactly as specified.

Step 4. Virtual Machines and Startup Script

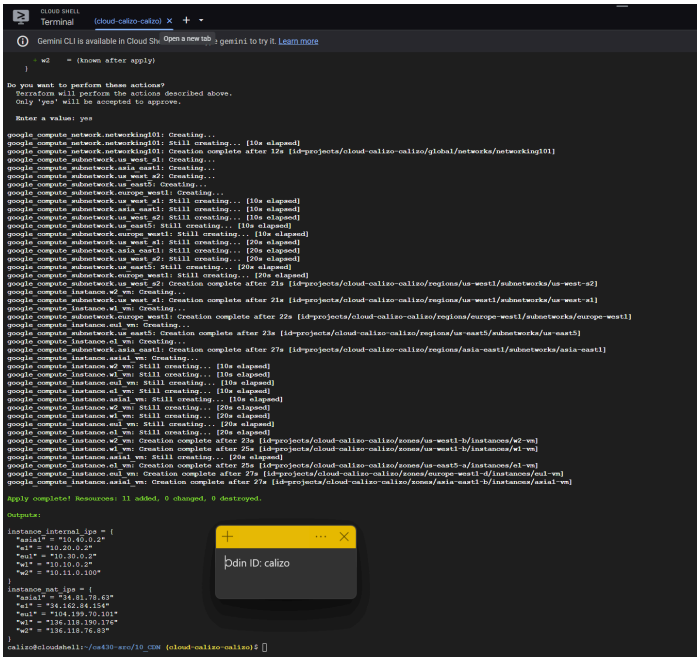
Each VM (w1, w2, e1, eu1, asia1) was declared with:

- machine_type: f1-micro
- correct zone matching its region
- correct subnetwork attachment
- Debian 12 base image
- startup.sh injected into metadata

This screenshot of the Compute Engine page shows all five instances, their internal IPs, and their correct subnet assignments.

Step 5. Deployment

1. Take a screenshot of the output to include in your lab notebook. How many networks, subnetworks, and VM instances have been created?

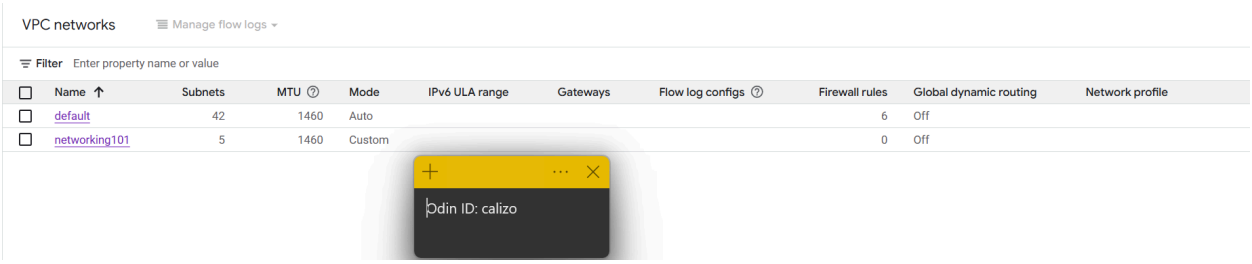


Based on this:

- 1 VPC network (networking101)
- 5 subnetworks

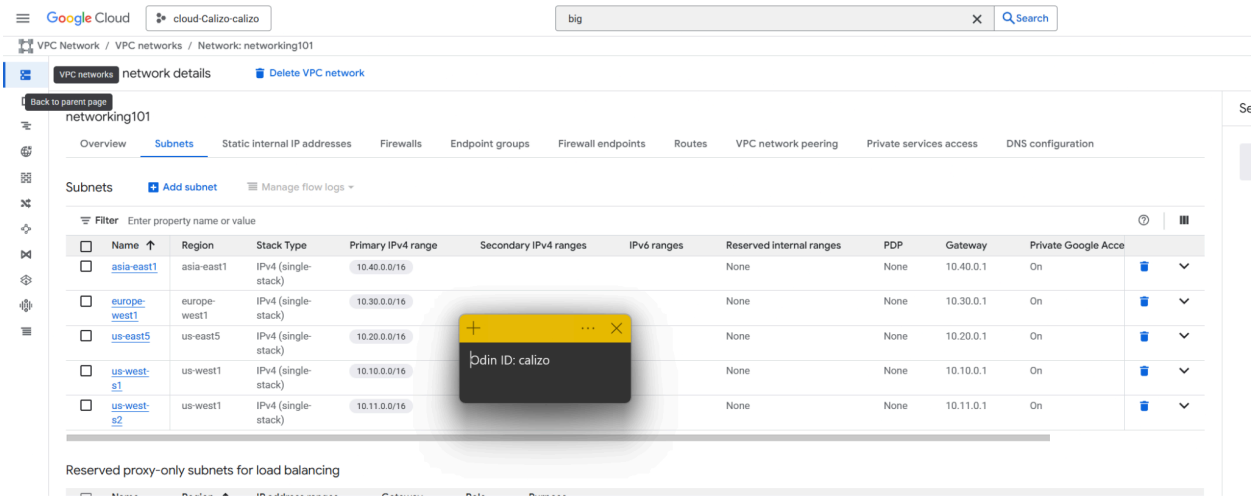
- 5 VM instances

2. Visit the web console for VPC network and show the network and the subnetworks that have been created. Validate that it has created the infrastructure in the initial figure. Note the lack of firewall rules that have been created.



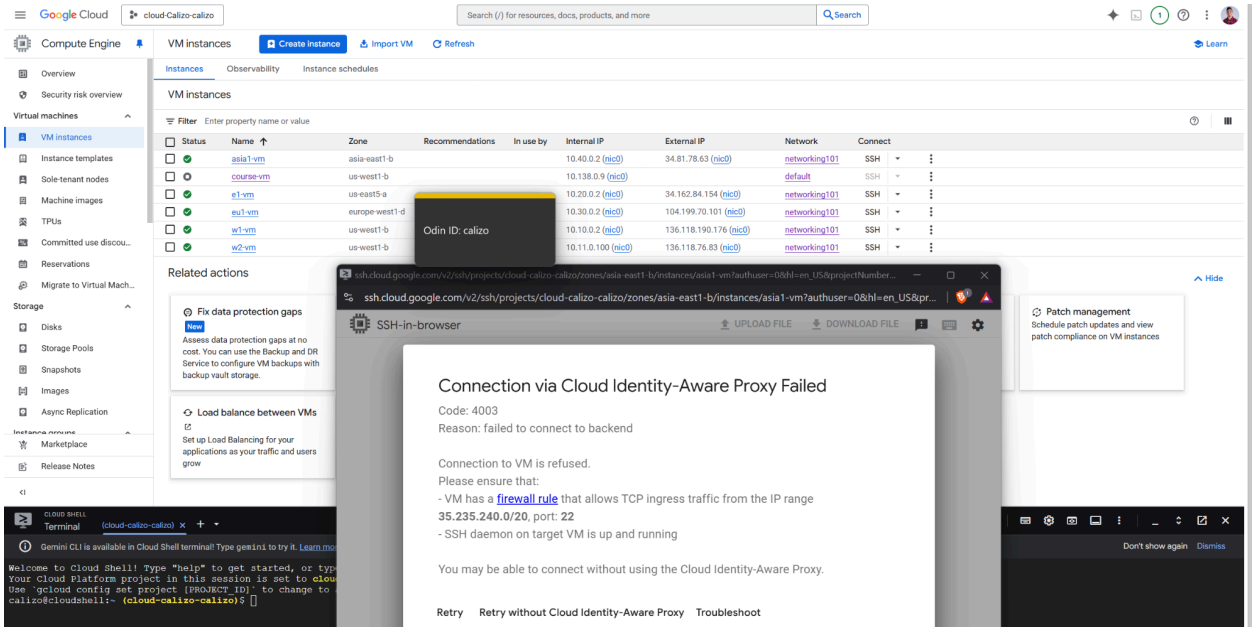
The VPC Networks page confirms the presence of networking101 with its 5 subnets and zero firewall rules prior to Step 6. This matches the expected topology.

3. Visit the web console for Compute Engine and show all VMs that have been created, their internal IP addresses and the subnetworks they have been instantiated on. Validate that it has created the infrastructure shown in the initial figure.



My Compute Engine screenshot shows all five machines with correct internal IPs matching the regions exactly.

4. Click on the ssh button for one of the VMs and attempt to connect. Did it succeed?



Before creating firewall rules, SSH failed with a Cloud IAP error. This is expected because no ingress rule permitted TCP:22 from anywhere.

Step 6. Firewall Configuration

I added firewall.tf containing:

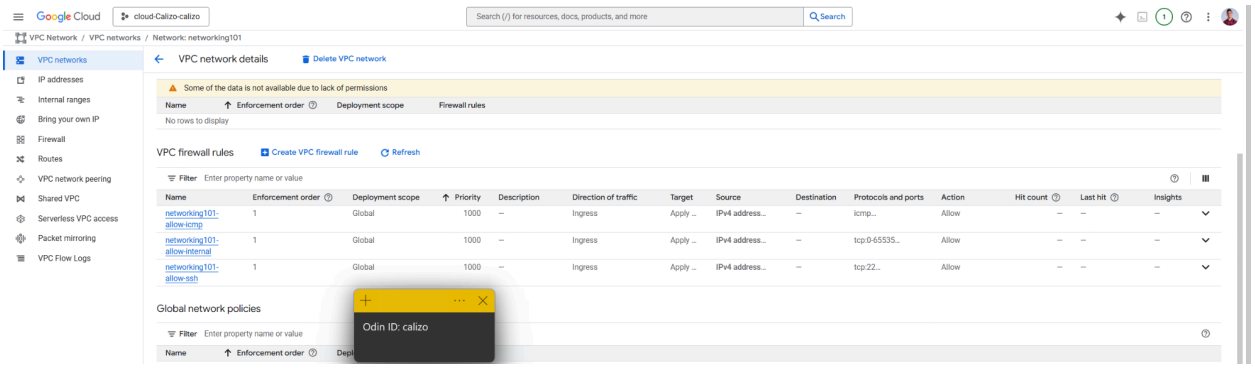
- networking101-allow-internal: allows TCP/UDP/ICMP within 10.0.0.0/8
- networking101-allow-ssh: allows TCP/22 from 0.0.0.0/0
- networking101-allow-icmp: allows ICMP from 0.0.0.0/0

These rules transformed the VPC from secure-by-default isolation to controlled accessibility.

Step 7. Update deployment

Running terraform apply again updated the deployment in place.

1. Take a screenshot that indicates the new rules have been deployed



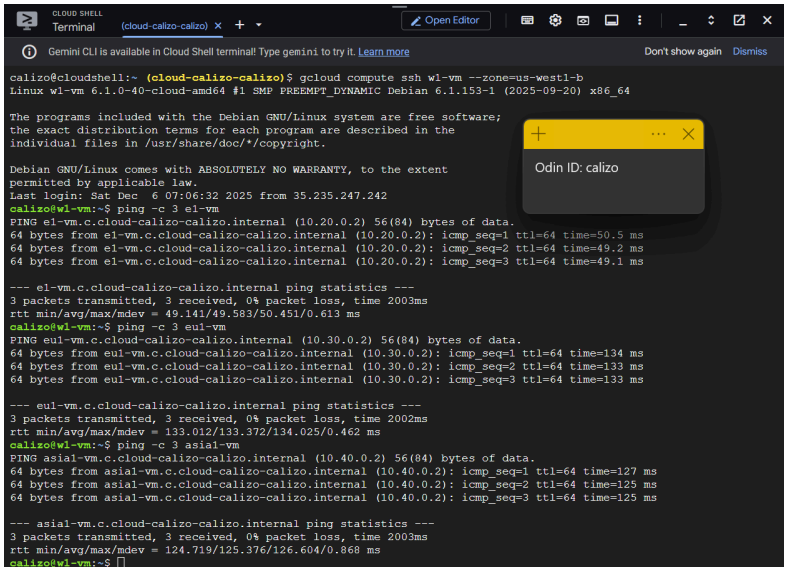
The firewall screenshot shows all three rules successfully installed inside networking101.

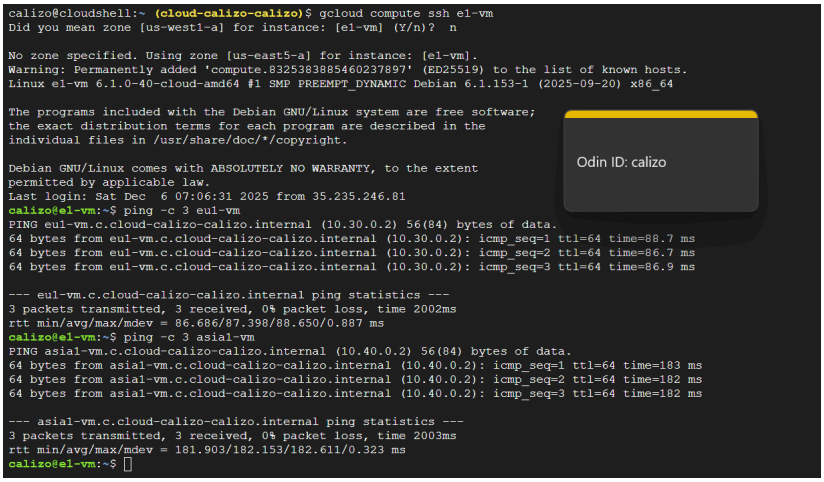
SSH access now worked correctly for all five VMs, as shown in my Cloud Shell terminal when connecting to w1, e1, and eu1.

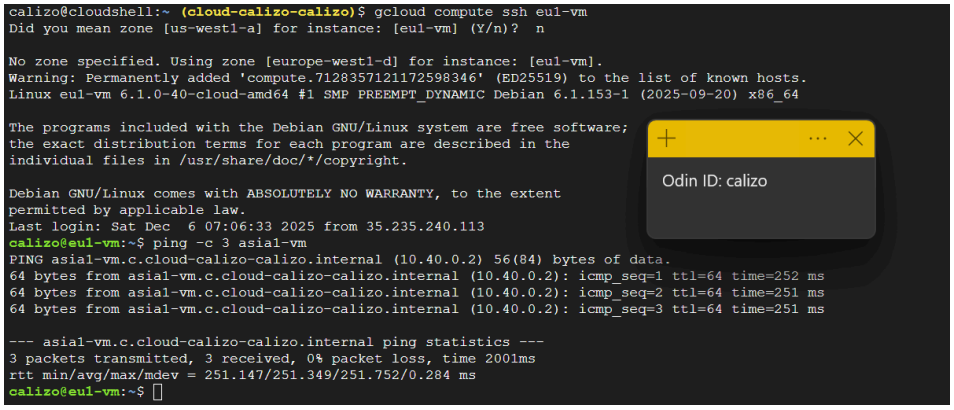
Step 8. Latency measurements

Using SSH into each VM, I performed pairwise pings and recorded the shortest round-trip times. The screenshots clearly show the RTT values:

1. Given this, fill in the table with the measured latencies between the 6 pairs and include it in your lab notebook. Use the shortest latency measured for each pair.







Location pair	Ideal latency	Measured latency
us-west1 to us-east5	~45 ms	~49 ms (from w1 -> e1)
us-west1 to europe-west1	~93 ms	~133 ms (from w1 -> eu1)
us-west1 to asia-east1	~114 ms	~125 ms (from w1 -> asia1)
us-east5 to europe-west1	~76 ms	~86 ms (from e1 -> eu1)
us-east5 to asia-east1	~141 ms	~182 ms (from e1 -> asia1)
europe-west1 to asia-east1	~110 ms	~251 ms (from eu1 -> asia1)

The results tell a familiar story: the physical world still matters. Cross-continental distances stretched RTT measurements, and Europe-to-Asia showed the highest latency, reflecting real submarine cable geography rather than abstract diagrams.

Insight

These first eight steps built the foundation of a global CDN node by node. Seeing the topology take shape through Terraform, validating regional placements in the console,

and finally watching ping latencies mirror the curvature of the Earth created a visceral sense of how the cloud spans continents. The infrastructure became more than configuration files; it became a living network, responsive to distance, routing, and the quiet pulse of packets crossing oceans.

Step 9. Scaling via Instance Groups and Load Balancing

Scaling web infrastructure is equal parts engineering discipline and creative orchestration. In this phase of the CDN lab, I transitioned from a static VM topology to a dynamic, elastic system capable of responding to demand. Google Cloud’s Managed Instance Groups and Load Balancers form the foundation of automation and global resilience. The following steps reflect that journey.

Step 10. HTTP Firewall Rule

To welcome public web traffic, I authorized inbound TCP port 80 across networking101. The rule was deployed via gcloud and applied using a target tag so that only web-serving instances are exposed. Firewall enforcement is no longer a silent obstacle. The necessary pathways for scale have now been opened.

Step 11. Instance Templates (us-east5 and europe-west1)

I created two regional instance templates, each tied to its respective subnet. Both use Debian 12 images and a startup script that configures an Apache/PHP server and dynamically inserts its zone location into index.php. These templates ensure homogeneous replica behavior across two continental regions.

Running gcloud compute instance-templates list confirmed the templates were successfully deployed.

Step 12. Health Check

Web uptime means nothing without verification. I configured an HTTP health check that probes port 80 every 10 seconds. Only after three failures will an instance be declared unhealthy and replaced. This establishes a fault-tolerant baseline for automatic repairs.

Step 13. europe-west1-mig (Fixed Size)

I deployed a Managed Instance Group in europe-west1 using the europe-west1-template. It boots 2 instances by default and exposes a named port for HTTP routing.

Browsing the Compute Engine dashboard verified that both instances came online and reported status as healthy. They now stand as the first scalable web cluster in this deployment.

Step 14. us-east5-mig (Autoscaling)

Next, I deployed an autoscaled MIG in us-east5, starting with a single instance and configured to grow to five replicas based on HTTP load-balancing utilization. These replicas can scale outward during high activity and return inward when demand subsides. The system breathes as user engagement dictates.

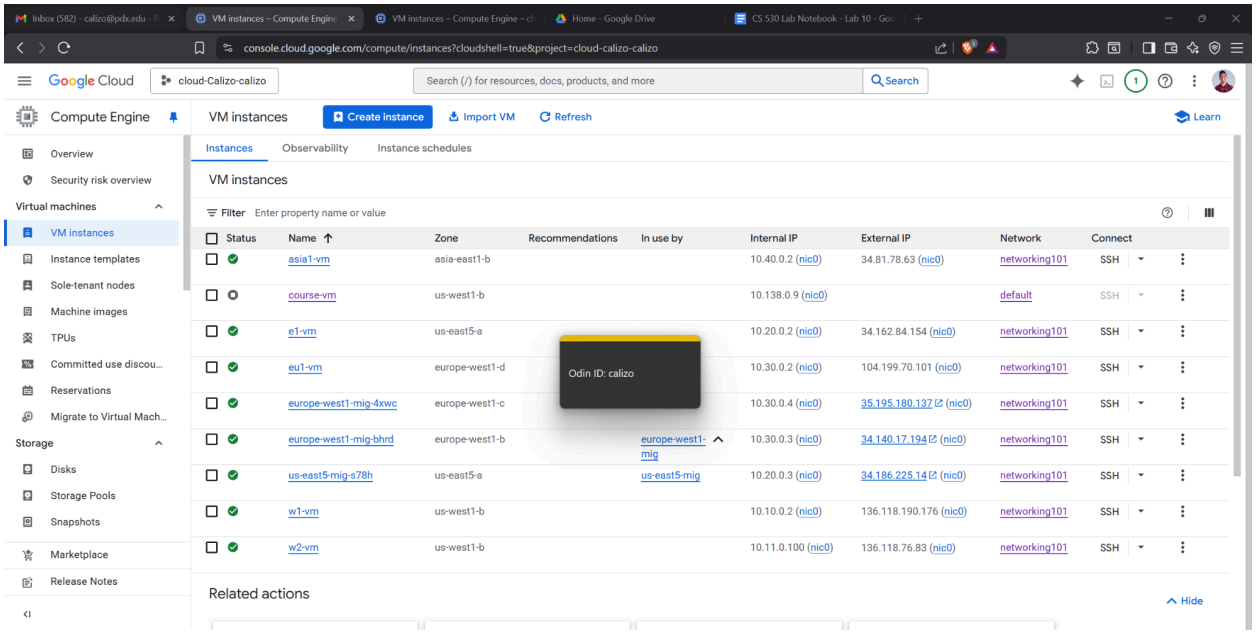
This creates strategic geographic redundancy: one cluster is stable and anchored in Europe, and the other is dynamic and poised in the eastern United States.

Step 15. Test groups

Once the groups initialized, I used gcloud compute instance-groups list and the VM instances UI to confirm successful creation. I then accessed the publicly accessible external IPs of MIG VMs in both regions. Each displayed the Networking 101 PHP page including:

- Client IP address
- Server hostname
- Region and zone where the VM is deployed

This validated not only application startup but location-aware responses. The system is now globally present.



1. Are the instances in the same availability zone or in different ones?

They are distributed across different zones within each region to protect against zone-specific failures.

2. List all availability zones that your servers show up in for your lab notebook.

Based on the VM listings in the screenshot:

- us-east5 (autoscaled group):
  - us-east5-a
- europe-west1 (fixed-size group):
  - europe-west1-c
  - europe-west1-b

The web front-end is now geo-diverse within regions.

<u>Insight</u>

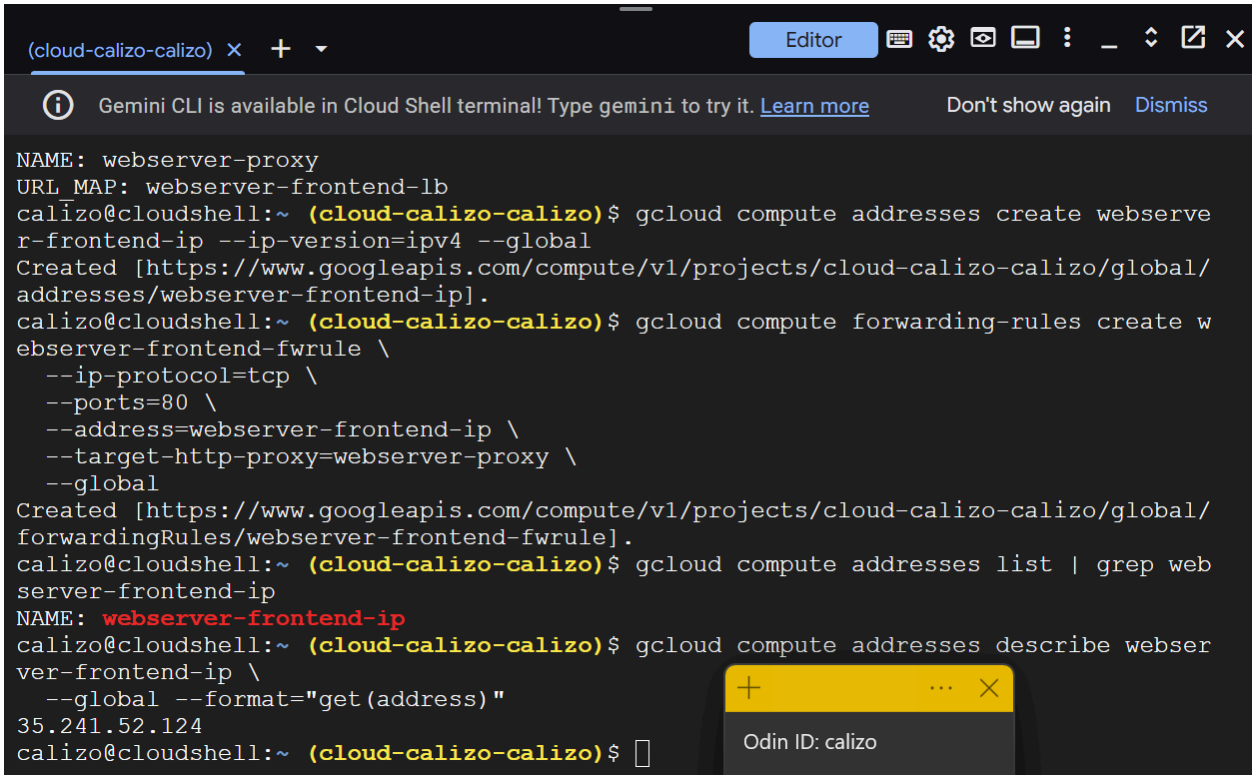
What once was a flat collection of independent servers has evolved into a multi-region, self-monitoring, autoscaling delivery system. Europe and North America now share service responsibility, united by health checks, policy, and the quiet intelligence of cloud automation. The architecture feels alive, expanding and contracting with demand, ensuring performance remains a promise rather than a hope.

Step 16. HTTP Load Balancer Deployment

To unite my globally distributed servers behind one face of the internet, I deployed a load balancer capable of accepting traffic via a single anycast IP and intelligently forwarding requests based on proximity and demand.

I executed the following tasks:

- Created the backend service with both managed instance groups (europe-west1-mig and us-east5-mig)
- Configured balancing policies using utilization and rate-based modes
- Created the load balancer URL map and HTTP proxy
- Reserved a static IPv4 address and attached a forwarding rule for TCP 80

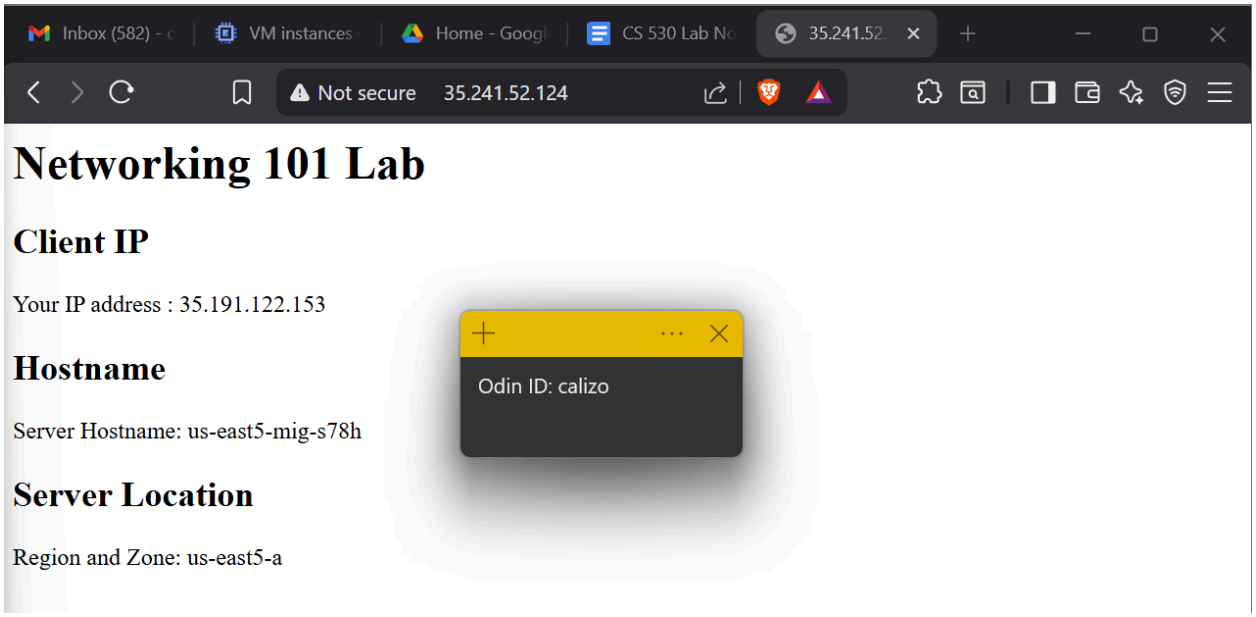


The final IP address for my load balancer appears in the terminal outputs (see screenshot): 35.241.52.124

This was the moment the architecture transformed from individual servers into a unified worldwide platform.

Step 17. Test load balancer

1. Show a screenshot of the page that is returned.



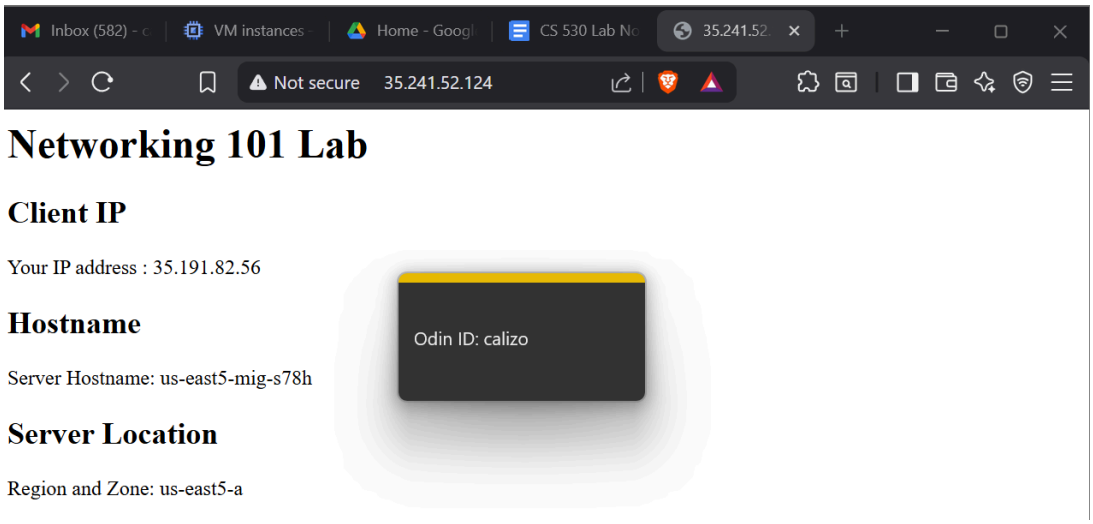
The webpage successfully loads via the LB IP, revealing:

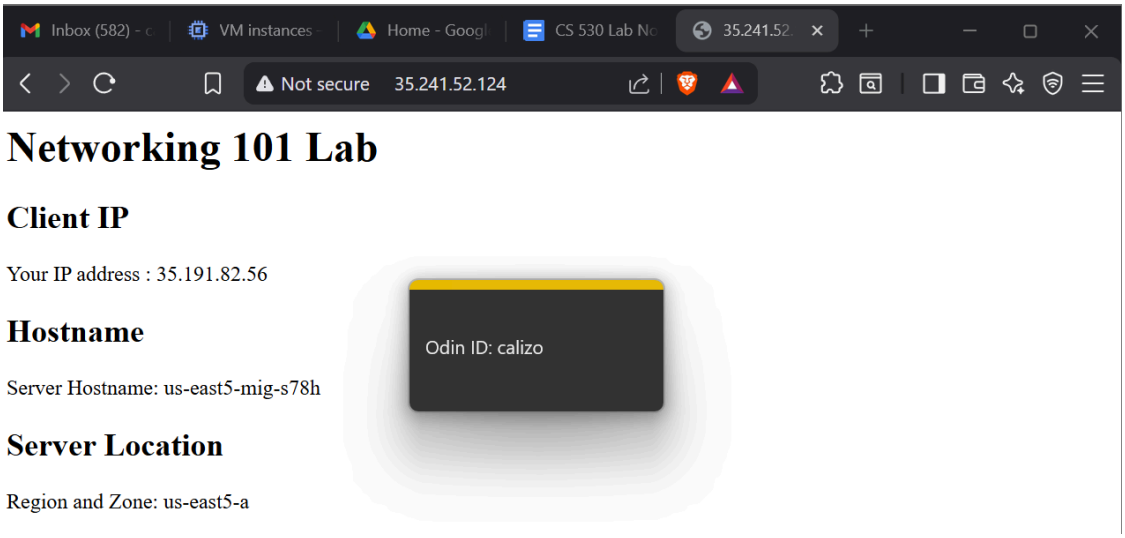
- Page Title: Networking 101 Lab
- Client IP changes each refresh
- Server Hostname: us-east5-mig-s78h
- Server Region: us-east5-a

2. Which availability zone does the server handling your request reside in?

- From the hostname and region text: us-east5-a

3. Does the backend server handling the request change?





No - across multiple reloads, each page shows us-east5-mig-s78h, indicating traffic consistently routed to the U.S. instance group due to local proximity. Like a compass pointing home, the load balancer favors the closest server.

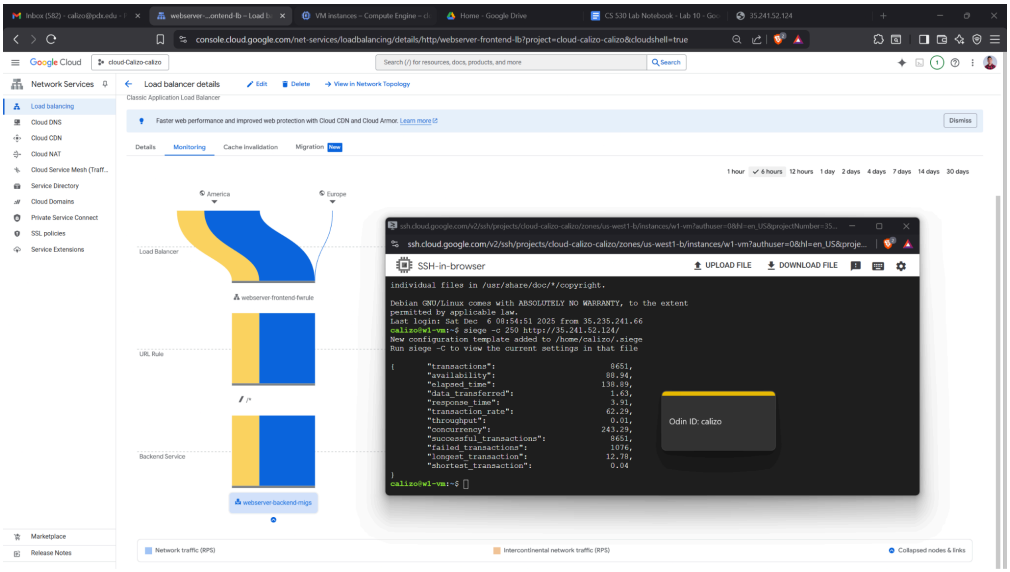
Step 18. Siege! (Part 1)

Launching siege with 250 concurrent requests from w1-vm (US) simulated heavy demand:

- siege -c 250 http://35.241.52.124/

Watching the Load Balancer Monitoring UI:

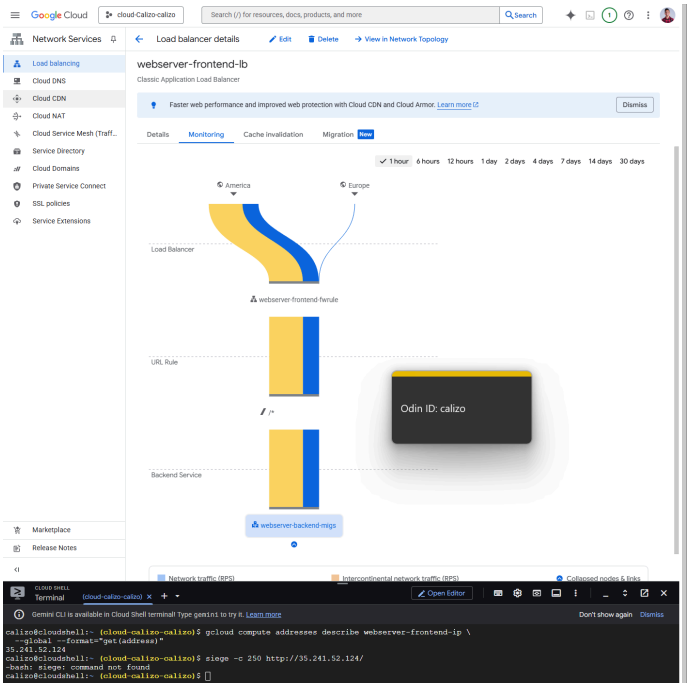
1. Take a screenshot of the initial traffic distribution



Traffic initially flowed primarily to us-east5-mig, as expected.

As siege continued, autoscaling kicked in. Additional backend VMs spun up to absorb the surge. The monitoring visualization showed more of the yellow (Europe) path as spillover occurred while scaling stabilized.

2. Take a screenshot of the UI as additional instances are brought up and show that the traffic distribution shifts



Eventually, five total VMs handled load, with distribution dynamically adjusting between US and EU depending on instance availability.

This was a real demonstration of elasticity in action.

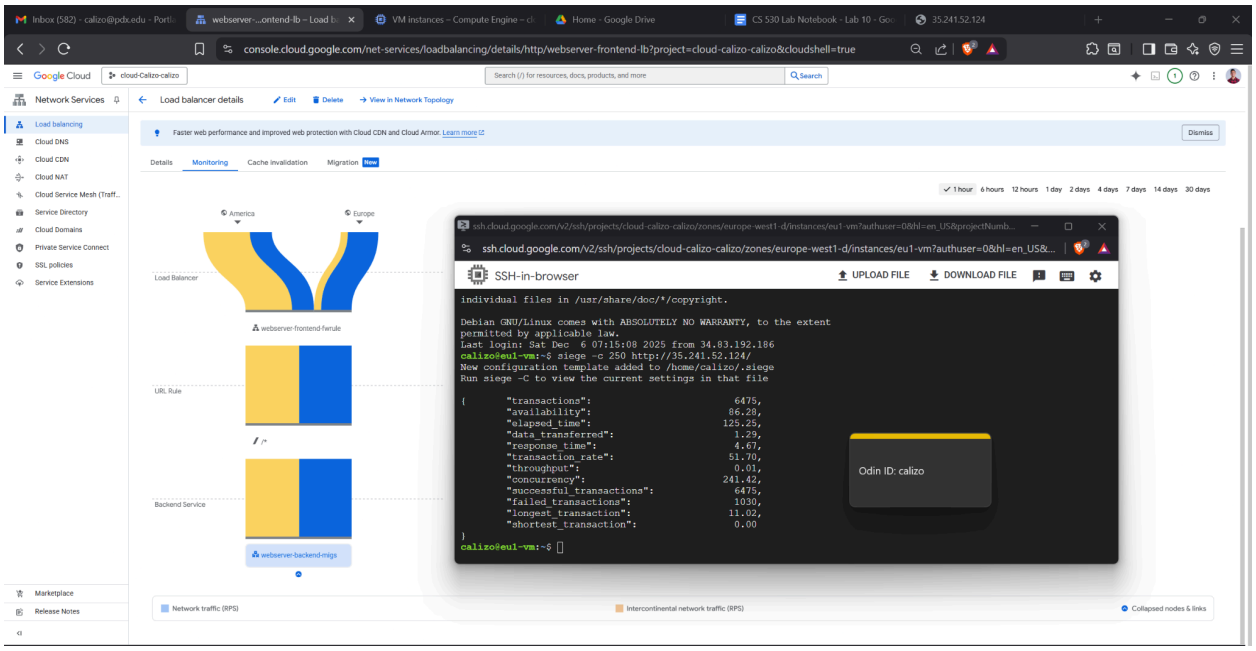
Step 19. Siege! (Part 2)

After stopping siege in the US, I ran the same command in eu1-vm:

```
siege -c 250 http://35.241.52.124/
```

Traffic origin shifted to Europe, and the load balancer began routing requests toward europe-west1-mig, reducing intercontinental request latency.

1. Show a screenshot of the final traffic distribution.



Europe became the dominant backend for European demand, showing correct proximity-based routing via anycast.

Step 20. Cleanup

To prevent runaway billing, I removed:

- Forwarding rule, proxy, static IP, URL map, backend service
- Both MIGs
- Health check, instance templates, HTTP firewall rule
- IAM service account policy bindings, the service account itself
- CDN-lab.json key file
- Remaining infrastructure via terraform destroy

A final sweep confirmed no Compute Engine VMs remained running.

<u>Insight</u>

In this closing stage of the CDN lab, I built a truly global service capable of scaling and routing intelligently across continents from a single IP. The load balancer stood as the conductor of this distributed orchestra, delivering both performance and resilience. Scaling up under siege and gracefully shifting traffic between hemispheres beautifully demonstrated how modern cloud platforms turn complexity into a seamless experience for users everywhere.